

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2024

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2024

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2024

I0I China candidate team papers / National training team 2024 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2024. Tập được dàn trang bằng Adobe InDesign; phần bìa và mục lục có lớp văn bản đọc được, nhưng nhiều trang thân bài trích xuất thành mojibake và sinh cảnh báo phong chữ. Bản LaTeX này chuyển ngữ phần nhận diện tập sách và toàn bộ mục lục, đồng thời dịch ba bài đầu tiên: bài của Zhou Kangyang về bài toán tính tổng hàm nhân tính, bài của Guo Yuchong về parity của matroid tuyến tính, và bài của Huang Luotian về hợp nhất–tách cấu trúc dữ liệu, dựa trên OCR và đối chiếu ảnh trang PDF.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2024*. Tập PDF gồm 424 trang và được tạo bằng Adobe InDesign. Phần văn bản trích xuất được không ghi riêng tên huấn luyện viên.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Một số tiến triển về bài toán tính tổng hàm nhân tính	Zhou Kangyang
11	Bàn về bài toán parity của matroid tuyến tính	Guo Yuchong
24	Bàn về hợp nhất và tách cấu trúc dữ liệu	Huang Luotian
37	Bàn về nhân tử Lagrange và đối ngẫu trong OI	Shi Kaicheng
54	Thuật toán liên quan tới bài toán cập nhật đoạn và truy vấn đoạn cùng ứng dụng	Shen Jiyu
69	Bàn về một số phương pháp giải bài toán quy hoạch động trong OI	Cao Li
85	Bàn về một số cách xử lý hợp nhất thông tin	Feng Zhengwei
100	Một số phương pháp cho bài toán đếm thứ tự topo trên cây	Chen Xinyang
121	Bàn về tối ưu hằng số trên phần cứng hiện đại	Song Jiaxing
171	Bàn về ứng dụng của một lớp phương pháp bao hàm–loại trừ và nghịch đảo	Dong Yiqi
182	Một lớp kỹ thuật dùng hàm sinh để mô tả thời điểm dừng của quá trình ngẫu nhiên	Ye Kai
196	Bàn về bài toán đồ thị đường trong thi tin học	Zhai Jincheng
206	Bàn về một lớp bài toán đoạn trên cây	Zhu Yifan
217	Báo cáo ra đề <i>Thế giới chìm trong cổ tích ngủ say</i>	Ye Liqi
230	Bài toán matching trên đồ thị có trọng số đỉnh	Ke Yisi
253	Bàn lại về đếm đường đi trên lưới	Wu Chang
277	Ứng dụng liên phân số và thuật toán Euclid trong OI	Ke Yijing
299	Bàn về độ phức tạp và ứng dụng của nó trong giải bài toán	Yang Minxing
315	Bàn về thuật toán duy trì một loại cặp số nguyên tố cùng nhau và ước chung lớn nhất	Du Guancheng
321	Bàn về lý thuyết tính toán và các bài toán khó trong OI	Yan Chenxiao
336	Bàn về ứng dụng thuật toán lattice trong phân tích đa thức hệ số nguyên	Xie Zihan
358	Bàn về bài toán tô màu danh sách của đồ thị	Zhong Zihou
367	Từ <i>Kén</i> : phân tích phương pháp quan sát trong thi tin học qua ví dụ	Li Jiayou
376	Bàn về q -analogue	Li Mingleyang
400	Bàn về chu trình tỉ lệ nhỏ nhất và ứng dụng	Wei Jiaze

Bài 1. Một số tiến triển về bài toán tính tổng hàm nhân tính

Zhou Kangyang, Trường Văn Uyên thuộc Tập đoàn Giáo dục Trung học Học Quân, Hàng Châu

Tóm tắt

Tính tổng hàm nhân tính là một lớp bài toán quan trọng trong số học. Với bài toán này, các nghiên cứu trước đây chủ yếu hướng tới giảm số lượng thừa số log trong độ phức tạp $\tilde{O}(n^{2/3})$.¹ Sau khi suy nghĩ, tác giả nhận thấy có thể đạt độ phức tạp thời gian $O(n^{1/2} \text{poly log } n)$.²

Bài viết này giới thiệu thuật toán đó và các tối ưu của nó.

¹Một nghiên cứu liên quan: <https://negiizhao.blog.uj.ac/blog/7165>.

²Bản đầu được viết ngày 2023-07-11 và công bố tại <https://www.cnblogs.com/zkyJuruo/p/17544928.html>.

1. Kiến thức chuẩn bị

1.1. Định nghĩa và quy ước

Định nghĩa 1.1. Hàm số học là hàm có miền xác định là tập số nguyên dương và miền giá trị là tập số phức.

Với một hàm số học f , ta quy ước

$$S_f(n) = \sum_{i=1}^n f(i).$$

Định nghĩa 1.2. Hàm nhân tính là một hàm số học $f(n)$ xác định trên các số nguyên dương, thỏa

$$f(1) = 1, \quad \gcd(a, b) = 1 \Rightarrow f(ab) = f(a)f(b).$$

Với một hàm nhân tính f , chỉ cần biết mọi giá trị $f(p^k)$ với $p \in \mathbb{P}, k \in \mathbb{Z}^+$, ta có thể xác định toàn bộ hàm f . Một số hàm nhân tính thường gặp có giá trị trên p^k như sau:

- hàm ϵ (hàm đơn vị), thỏa $\epsilon(p^k) = 0$;
- hàm μ , thỏa $\mu(p^k) = -[k = 1]$;
- hàm φ , thỏa $\varphi(p^k) = (p - 1)p^{k-1}$;
- hàm I , thỏa $I(p^k) = 1$;
- hàm id_t , thỏa $\text{id}_t(p^k) = p^{kt}$.

Định nghĩa 1.3. Với hai hàm số học f, g , định nghĩa tích chập Dirichlet của chúng là

$$(f * g)(x) = \sum_{d|x} f(d)g\left(\frac{x}{d}\right).$$

Nếu xem tích chập Dirichlet là phép nhân của các hàm số học, và phép cộng trực tiếp của hàm là phép cộng, thì phép cộng và phép nhân trên các hàm số học thỏa giao hoán, kết hợp và phân phối.

Định lý 1.1. Tích chập Dirichlet h của hai hàm nhân tính f, g cũng là một hàm nhân tính.

Chứng minh. Chỉ cần chứng minh với mọi $\gcd(x, y) = 1$ ta có $h(xy) = h(x)h(y)$:

$$\begin{aligned} h(xy) &= \sum_{d|xy} f(d)g\left(\frac{xy}{d}\right) \\ &= \sum_{u|x, v|y} f(uv)g\left(\frac{xy}{uv}\right) \\ &= \sum_{u|x} f(u)g\left(\frac{x}{u}\right) \sum_{v|y} f(v)g\left(\frac{y}{v}\right) \\ &= h(x)h(y). \end{aligned}$$

1.2. Powerful Number

Định nghĩa 1.4. Với một số nguyên dương m , nếu với mọi thừa số nguyên tố p của m đều có $p^2 | m$, thì gọi m là một *Powerful Number*.

Trong phần sau, ta viết tắt *Powerful Number* là PN. Trong công thức, PN biểu thị tập tất cả các PN.

Định lý 1.2. Mọi PN đều có thể biểu diễn dưới dạng a^2b^3 , trong đó a, b là các số nguyên dương.

Chứng minh. Giả sử

$$x = \prod_i p_i^{\alpha_i}$$

là một PN. Chỉ cần biểu diễn từng $p_i^{\alpha_i}$ dưới dạng $a_i^2b_i^3$, rồi lấy $a = \prod_i a_i, b = \prod_i b_i$.

Nếu α_i lẻ, lấy

$$a_i = p_i^{(\alpha_i-3)/2}, \quad b_i = p_i;$$

nếu α_i chẵn, lấy

$$a_i = p_i^{\alpha_i/2}, \quad b_i = 1.$$

Định lý 1.3. Số lượng PN không vượt quá n là $O(\sqrt{n})$.

Chứng minh. Biểu diễn mọi PN không vượt quá n dưới dạng a^2b^3 . Khi liệt kê a , số PN không vượt quá

$$\sum_{a=1}^{\sqrt{n}} \left(\frac{n}{a^2} \right)^{1/3}.$$

Tổng này có thể ước lượng bởi

$$\int_1^{\sqrt{n}} \left(\frac{n}{x^2} \right)^{1/3} dx = n^{1/3} \int_1^{\sqrt{n}} x^{-2/3} dx = n^{1/3} (3n^{1/6} - 3),$$

tức ở cấp $O(\sqrt{n})$.

Sàng tuyến tính mọi số nguyên tố không vượt quá $\sqrt{n}$, rồi DFS trên các số nguyên tố sẽ tìm được mọi PN không vượt quá n . Độ phức tạp bằng số PN, tức $O(\sqrt{n})$.

Ví dụ 1.1 (phần $n = 1$ của UOJ Long Round 7, Mã kiểm tra). Cho c . Hàm nhân tính $q(x)$ thỏa

$$q(p^k) = p^{c\lfloor k/2 \rfloor}.$$

Tính $\sum_{i=1}^m q(i)$, đáp án lấy modulo 2^{32} , với $m \leq 1.2 \times 10^{11}$.

Thiết kế hàm nhân tính $f(x)$ thỏa $f(p) = q(p)$, và dựng hàm nhân tính g thỏa $g * f = q$. Với số nguyên tố p ,

$$g(p)f(1) + g(1)f(p) = q(p),$$

suy ra $g(p) = 0$. Vì vậy g chỉ có giá trị khác không tại các PN.

Ta có

$$\begin{aligned} \sum_{i=1}^m q(i) &= \sum_{i=1}^m \sum_{jk=i} g(j)f(k) \\ &= \sum_{\substack{j \in PN \\ j \leq m}} g(j) \sum_{k=1}^{\lfloor m/j \rfloor} f(k) \\ &= \sum_{\substack{j \in PN \\ j \leq m}} g(j) \left\lfloor \frac{m}{j} \right\rfloor. \end{aligned}$$

Tiền xử lý $g(p^k)$ cho từng p^k . Từ

$$q(p^k) = \sum_{i=0}^k f(p^i)g(p^{k-i})$$

có thể giải $g(p^k)$ theo thứ tự tăng dần của k , đồng thời duy trì giá trị hàm g trong lúc DFS tìm PN. Như vậy bài toán được giải trong $O(\sqrt{m})$.

1.3. Sàng Du Jiao

Với một số hàm nhân tính đặc biệt như μ và φ , ta có thể xây dựng tích chập Dirichlet để tính tổng tiền tố trong thời gian $O(n^{2/3})$. Trong cộng đồng thi tin học Trung Quốc, thuật toán kiểu này được gọi là “sàng Du Jiao”.

Với hàm μ , ta có $\mu * I = \epsilon$.

Xét bài toán tổng quát hơn. Nếu có các hàm số học A, B, C thỏa

$$A(1) = B(1) = C(1) = 1, \quad A * B = C,$$

và ta tính nhanh được các giá trị điểm của S_B, S_C , thì làm sao tính $S_A(n)$?

Chú ý rằng

$$\begin{aligned} \sum_{i=1}^n C(i) &= \sum_{i=1}^n \sum_{jk=i} A(j)B(k) \\ &= \sum_{k=1}^n B(k) \sum_{j=1}^{\lfloor n/k \rfloor} A(j) \\ &= S_A(n) + \sum_{k=2}^n B(k) S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right). \end{aligned}$$

Do đó

$$S_A(n) = S_C(n) - \sum_{k=2}^n B(k) S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right),$$

có thể tính $S_A(n)$ bằng đệ quy. Lại có

$$\left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor,$$

nên trong quá trình đệ quy, các giá trị thật sự cần tính chỉ là

$$S_A\left(\left\lfloor \frac{n}{k} \right\rfloor\right) \quad (k \in \mathbb{Z}^+).$$

Định lý 1.4. Với số nguyên dương n , tập

$$\left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid i \in \mathbb{Z}^+ \right\}$$

có kích thước $O(\sqrt{n})$.

Chứng minh. Với $k \leq \sqrt{n}$, chỉ có $\sqrt{n}$ giá trị $\lfloor n/k \rfloor$, hiển nhiên là $O(\sqrt{n})$ loại. Với $k > \sqrt{n}$, ta có $\lfloor n/k \rfloor \leq \sqrt{n}$, nên cũng chỉ có $O(\sqrt{n})$ loại.

Vì vậy ta chỉ cần tính $O(\sqrt{n})$ giá trị điểm của S_A .

Khi tính một $S_A(m)$, gom các đoạn liên tiếp có cùng giá trị $\lfloor m/k \rfloor$ lại với nhau (tức chia khối theo phép chia nguyên), ta có thể đệ quy tới các bài toán con trong $O(\sqrt{m})$.

Với $m \leq n^{2/3}$, ta xử lý trước $n^{2/3}$ giá trị điểm đầu của A . Với $m > n^{2/3}$, dùng công thức đệ quy ở trên. Phần này có độ phức tạp

$$O\left(\sum_{i=1}^{n^{1/3}} \sqrt{\frac{n}{i}}\right) = O\left(\int_1^{n^{1/3}} \sqrt{\frac{n}{x}} dx\right) = O(n^{2/3}).$$

Tổng độ phức tạp là $O(n^{2/3})$ cộng với phần xử lý trước $O(n^{2/3})$ giá trị đầu của A . Với μ và φ , phần xử lý trước này đều làm được trong $O(n^{2/3})$.

Định nghĩa 1.5. Định nghĩa sàng khối của hàm số học f đối với n là các giá trị điểm của $S_f(n)$ trên tập

$$\left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid i \in \mathbb{Z}^+ \right\}.$$

Trong quá trình trên, ta dùng sàng khối của B và C đối với n để tìm sàng khối của A đối với n . Hiển nhiên, nếu biết sàng khối của A và B đối với n , ta cũng có thể tìm sàng khối của C đối với n . Ta gọi C là **tích chập sàng khối** của A và B . Các phép toán trên sàng khối vẫn thỏa giao hoán, kết hợp và phân phối.

2. Tích chập sàng khối

Với các hàm số học A, B, C thỏa

$$A(1) = B(1) = C(1) = 1, \quad A * B = C,$$

sàng Du Jiao ở trên đã dùng sàng khối của B, C đối với n để giải sàng khối của A đối với n .

Ở đây trước hết xét bài toán đơn giản hơn: đã biết sàng khối của A, B đối với n , hãy tính sàng khối của $C = A * B$ đối với n . Bài toán này thực ra có thể giải trong $O(n^{1/2} \text{poly log } n)$. Trong phần dưới, mặc định sàng khối là sàng khối đối với n .

Ta cần tính các tổng tiền tố trên mọi $\lfloor n/t \rfloor$ của

$$C(z) = \sum_{xy=z} A(x)B(y).$$

Trước hết xét trường hợp $x > \sqrt{n}$; trường hợp $y > \sqrt{n}$ là đối xứng. Nếu $xy \leq \lfloor n/t \rfloor$ thì $x \leq n/(ty)$. Do đó khi liệt kê t, y , các x có đóng góp là một đoạn tiền tố. Phần này có độ phức tạp $O(\sqrt{n} \log n)$.

Như vậy chỉ còn cần xử lý trường hợp $x \leq \sqrt{n}$ và $y \leq \sqrt{n}$. Điều kiện $xy \leq n/t$ tương đương

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right).$$

Ta làm một ước lượng như sau. Chọn một độ dài khối nguyên S với $3 \leq S \leq n$, và cho rằng

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right)$$

khi và chỉ khi

$$\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor \leq S \ln \left(\frac{n}{t} \right).$$

Ước lượng này có thể làm bằng nhân đa thức. Thiết kế hai đa thức $F(z), G(z)$ sao cho $[z^k]F(z)$ là tổng các $A(x)$ thỏa $\lfloor S \ln x \rfloor = k$, và $[z^k]G(z)$ là tổng các $B(x)$ thỏa $\lfloor S \ln x \rfloor = k$. Tính $H(z) = F(z)G(z)$, khi đó hệ số $[z^k]H(z)$ là tổng $A(x)B(y)$ với $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor = k$. Chỉ cần làm một phép chập độ dài $S \ln n$.

Nhưng làm như vậy hiển nhiên không đúng hoàn toàn. Nó chỉ sai khi

$$\ln x + \ln y \leq \ln \left(\frac{n}{t} \right) \quad \text{và} \quad \lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor > S \ln \left(\frac{n}{t} \right).$$

Khi đó

$$S \ln x + S \ln y \in \left(S \ln \left(\frac{n}{t} \right) - 2, S \ln \left(\frac{n}{t} \right) \right],$$

nên

$$\ln x + \ln y + \ln t \in \left(\ln n - \frac{2}{S}, \ln n \right],$$

tức

$$xyt \in (ne^{-2/S}, n].$$

Trong đó $e^{-2/S}$ ở cấp $1 - O(1/S)$. Vì vậy xyt nằm trong một đoạn có độ dài $O(n/S)$.

Giả sử đoạn này là $[n - L, n]$. Dùng sàng đoạn để lấy phân tích thừa số nguyên tố của mọi số trong đoạn. Cụ thể hơn, trước hết sàng mọi số nguyên tố nhỏ không vượt quá $\sqrt{n}$. Với mỗi số nguyên tố, ta tìm được các bội của nó trong $[n - L, n]$, từ đó sàng ra mọi thừa số nguyên tố $\leq \sqrt{n}$ của các số trong đoạn. Một số có nhiều nhất một thừa số nguyên tố $> \sqrt{n}$, nên chỉ cần xét phần còn lại sau khi chia hết các số nguyên tố nhỏ.

Sau khi có phân tích thừa số nguyên tố, DFS trên các thừa số nguyên tố sẽ nhanh chóng tìm được mọi bộ (x, y, t) có thể. Với một bộ (x, y, t) , kiểm tra trong $O(1)$ xem nó có bị ước lượng sai hay không. Với một $v \in [n - L, n]$, thời gian đóng góp là $d_3(v)$, trong đó

$$d_3(v) = \sum_{xyz=v} 1.$$

Theo kết quả trong số học giải tích [1],

$$\sum_{i \leq n} d_3(i) = nP_3(\ln n) + O(n^{43/96+\epsilon}),$$

nên

$$\sum_{i=n-L}^n d_3(i) = LP_3(\ln n) + O(n^{43/96+\epsilon}),$$

trong đó P_3 là một đa thức bậc hai. Bản thân đầu ra của thuật toán đã có $O(\sqrt{n})$, lớn hơn $O(n^{43/96+\epsilon})$, nên có thể bỏ qua phần độ phức tạp này. Phần còn lại có độ phức tạp

$$O(L \log^2 n) = O\left(\frac{n}{S} \log^2 n\right).$$

Tổng độ phức tạp là

$$O\left(S \log^2 n + \frac{n}{S} \log^2 n\right).$$

Lấy $S = \sqrt{n}$ thu được độ phức tạp thời gian

$$O(\sqrt{n} \log^2 n).$$

3. Tính tổng hàm nhân tính

Với một hàm nhân tính f thỏa một số tính chất đặc biệt, chẳng hạn

$$f(p^k) = p^k(p^k - 1),$$

làm sao tính sàng khối của f ?

Trước hết, giá trị $f(p^k)$ với $k \geq 2$ không quá quan trọng. Nếu có một hàm nhân tính khác g thỏa $f(p) = g(p)$, và tổng tiền tố của g dễ tính, thì chỉ cần thiết kế hàm nhân tính h thỏa $h * g = f$. Theo kết luận ở trên, h chỉ có giá trị tại PN, và các giá trị đó có thể tính trong $O(\sqrt{n})$. Vì vậy dễ dàng tính sàng khối của h trong $O(\sqrt{n})$, rồi dùng tích chập sàng khối của h và g để tính sàng khối của f .

Ta trước hết thiết kế một hàm số học q chỉ khác không tại các số nguyên tố và thỏa $q(p) = f(p)$.

Với một hàm số học q có $q(1) = 0$, định nghĩa

$$\exp(q) = \sum_{i=0}^{\infty} \frac{q^i}{i!},$$

trong đó phép chập là tích chập sàng khối. Xét $d = \exp(q)$. Với

$$x = \prod_{i=1}^k p_i^{\alpha_i},$$

hàm $d(x)$ chỉ có đóng góp từ hạng

$$q^{\sum_{i=1}^k \alpha_i},$$

và giá trị bằng

$$\frac{\prod_{i=1}^k q(p_i)^{\alpha_i}}{(\sum_{i=1}^k \alpha_i)!} \binom{\sum_{i=1}^k \alpha_i}{\alpha_1, \alpha_2, \dots, \alpha_k} = \frac{\prod_{i=1}^k q(p_i)^{\alpha_i}}{\prod_{i=1}^k \alpha_i!}.$$

Do đó d là một hàm nhân tính và

$$d(p^k) = \frac{f(p)^k}{k!},$$

đặc biệt $d(p) = f(p)$. Như vậy, nếu tính được sàng khối của q , ta có thể dùng $\log n$ lần tích chập sàng khối (khi $i > \log n$, q^i không còn giá trị trong n hạng đầu) để tính sàng khối của d , từ đó suy ra sàng khối của f .

Làm sao tính sàng khối của q ? Xét một ý tưởng tương tự sàng Min_25. Tách hàm q thành tổng có trọng số của vài q_i , rồi dùng sàng khối của các q_i để tính sàng khối của q . Ví dụ, nếu

$$q(p) = p(p-1),$$

ta có thể tách thành

$$q_1(p) = p^2, \quad q_2(p) = p, \quad q = q_1 - q_2.$$

Khi $q(p)$ là đa thức bậc hằng theo p , ta đều có thể tách như vậy.

Làm sao tính sàng khối của q_i ? Ta làm ngược lại. Lấy $q_1(p) = p^2$ làm ví dụ. Với hàm nhân tính d thỏa

$$d(p^k) = q_1(p)^k,$$

ta có $d(x) = x^2$, nên sàng khối rất dễ tính. Với hàm nhân tính d thỏa

$$d(p^k) = \frac{q_1(p)^k}{k!},$$

do nó có cùng giá trị tại các điểm p với hàm trước, ta có thể dùng PN để chuyển đổi lẫn nhau trong độ phức tạp của một lần tích chập sàng khối. Vì vậy sàng khối của nó cũng tính được.

Hàm d thỏa $d(p^k) = q_1(p)^k/k!$ chính là $\exp(q_1)$. Điều này gợi ý dùng một ý tưởng tương tự $\ln$ để suy ngược q_1 từ d .

Với một hàm số học d có $d(1) = 1$, định nghĩa

$$\ln(d) = \sum_{i=1}^{\infty} \frac{(-1)^{i-1} (d - \epsilon)^i}{i}.$$

Ta có $q_1 = \ln(d)$.

Chứng minh. Chỉ cần lần lượt chứng minh $\exp(\ln(d)) = d$ và hàm q thỏa $\exp(q) = d$ là duy nhất.

Trước hết chứng minh $\exp(\ln(d)) = d$:

$$\exp(\ln(d)) = \sum_{j=0}^{\infty} \frac{\left(\sum_{i=1}^{\infty} \frac{(-1)^{i-1} (d - \epsilon)^i}{i} \right)^j}{j!}.$$

Dùng tính phân phối và kết hợp của sàng khối để khai triển, ta sẽ nhận được biểu thức dạng $\sum_i a_i d^i$. Có thể dùng đa thức $A(z)$ để mô tả nó, trong đó $[z^k]A(z) = a_k$. Định nghĩa của $\exp$ và $\ln$ trong tích chập sàng khối tương ứng với đa thức:

$$\exp(z) = \sum_{i=0}^{\infty} \frac{z^i}{i!}, \quad \ln(1+z) = \sum_{i=1}^{\infty} \frac{(-1)^{i-1} z^i}{i}.$$

Vì $A(z) = \exp(\ln z) = z$, nên $a_i = [i = 1]$, do đó $\exp(\ln(d)) = d$.

Nếu $\exp(q) = d$, tức

$$\epsilon + q + \sum_{i=2}^{\infty} \frac{q^i}{i!} = d,$$

ta có thể xác định lần lượt giá trị $q(i)$ theo thứ tự tăng dần. Đặt

$$p = \sum_{i=2}^{\infty} \frac{q^i}{i!},$$

khi đó $p(i)$ chỉ liên quan tới $q(j)$ với $j < i$, nên có thể dùng

$$q(i) = d(i) - [i = 1] - p(i)$$

để giải $q(i)$. Do đó q được xác định duy nhất.

Trong công thức $\ln$ ở trên cũng chỉ cần liệt kê $i \leq \log n$, vì vậy cũng chỉ cần $\log n$ lần tích chập sàng khối.

Do đó thuật toán này có độ phức tạp thời gian

$$O(\sqrt{n} \log^3 n).$$

4. Tối ưu

Dù ta đã có một thuật toán $O(n^{1/2} \text{poly log } n)$, nó vẫn quá chậm để dùng thực tế trong OI. Có tối ưu được không?

Dưới đây là một số hướng tối ưu đơn giản, có thể hạ độ phức tạp thời gian của thuật toán xuống

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

4.1. Tối ưu exp

Khi tính $\exp(f)$, với f chỉ có giá trị tại các số nguyên tố, ta phải làm $\log n$ lần tích chập sàng khối; điều này khá lãng phí. Xét cách tối ưu.

Trước hết tách f thành phần có chỉ số $\leq \sqrt{n}$, gọi là f_0 , và phần có chỉ số $> \sqrt{n}$, gọi là f_1 . Khi đó

$$\exp(f) = \exp(f_0 + f_1) = \exp(f_0) \exp(f_1).$$

Vì $\exp(f_1) = \epsilon + f_1$ dễ tính, phần khó là $\exp(f_0)$.

Khi tính $\exp(f_0)$, $\sqrt{n}$ hạng đầu của kết quả cuối có thể tính trực tiếp; khó khăn nằm ở phần $> \sqrt{n}$.

Xét cách tính trực tiếp bằng phương pháp lấy phần nguyên rồi sửa sai ở trên. Chọn số nguyên S , và cộng $f_{0,i}$ vào vị trí $v_{[S \ln i]}$. Sau đó trực tiếp lấy $\exp$ đa thức theo v .

Khi nào cách này sai? Giả sử

$$m = \prod_{i=1}^k p_i^{\alpha_i}.$$

Nếu vị trí của m bị ước lượng sai, đặt $t = \lfloor n/m \rfloor$, thì

$$\sum_{i=1}^k \alpha_i \lfloor S \ln p_i \rfloor \in \left(S \ln \left(\frac{n}{t} \right), S \ln \left(\frac{n}{t} \right) + \sum_{i=1}^k \alpha_i \right).$$

Suy ra

$$\sum_{i=1}^k \alpha_i \ln p_i + \ln t - \ln n \in \left[-\frac{\sum_{i=1}^k \alpha_i}{S}, 0 \right],$$

tức

$$\ln(mt) \in \left[\ln n - \frac{\sum_i \alpha_i}{S}, \ln n \right].$$

Đây là một đoạn có độ dài

$$O\left(\frac{n \log n}{S}\right),$$

vì vậy chỉ các mt trong đoạn này mới bị ước lượng sai.

Dùng sàng đoạn để sàng mọi thừa số nguyên tố $\leq \sqrt{n}$ trong đoạn đó. Sau khi sàng ra thừa số nguyên tố, DFS trên các thừa số nguyên tố sẽ nhanh chóng tìm các cặp (m, t) ; trong lúc DFS cũng có thể đồng thời tìm giá trị hàm tại m . Độ phức tạp DFS cùng cấp với số cặp.

Số cặp không vượt quá

$$O\left(\sum_{m=1}^{\sqrt{n}} \frac{n \log n}{Sm}\right) = O\left(\frac{n \log^2 n}{S}\right).$$

Phần đa thức phía trước có độ phức tạp $O(S \log^2 n)$. Lấy $S = \sqrt{n}$ thu được độ phức tạp $O(\sqrt{n} \log^2 n)$.

Ta còn có thể chọn một ngưỡng L và xử lý riêng các thừa số nguyên tố $\leq L$. Khi tính $\exp(f_0)$, không xét các thừa số $\leq L$, tức đặt chúng về 0 ở các vị trí tương ứng trong f_0 . Sau khi tính xong, thêm lần lượt các thừa số nguyên tố đó vào.

Khi thêm số nguyên tố p , giả sử đáp án trước đó là d , thì đáp án mới là

$$d'(n) = \sum_{k=0}^{\log_p n} f(p^k) d\left(\left\lfloor \frac{n}{p^k} \right\rfloor\right).$$

Chi phí cần thiết là $\sqrt{n} \log_p n$. Tổng chi phí thêm vào là

$$\sum_{\substack{p \leq L \\ p \in \mathbb{P}}} \frac{\sqrt{n} \log n}{\log p} = O\left(\frac{L \sqrt{n} \log n}{\log L}\right).$$

Sau khi loại bỏ các thừa số nguyên tố $\leq L$, mỗi m ở trên chỉ còn thừa số nguyên tố $> L$. Nếu

$$m = \prod_{i=1}^k p_i^{\alpha_i},$$

thì

$$\sum_{i=1}^k \alpha_i \leq \log_L n.$$

Vì vậy độ dài đoạn có thể ước lượng sai giảm xuống

$$O\left(\frac{n \log n}{S \log L}\right).$$

Khi đó độ phức tạp tính $\exp(f_0)$ trở thành

$$O\left(\frac{n \log n \log_L n}{S} + S \log^2 n\right).$$

Lấy

$$S = \sqrt{\frac{n}{\log L}},$$

ta được độ phức tạp

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log L}}\right).$$

Lấy $L = \log n$, độ phức tạp thời gian là

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

4.2. Tối ưu ln

Khi tính ln, ta cần biến một hàm nhân tính thỏa

$$f(p^k) = \frac{f(p)^k}{k!}$$

thành một hàm số học g chỉ có giá trị tại p và thỏa $g(p) = f(p)$.

Từ tính chất ở trên có thể trực tiếp lấy $\sqrt{n}$ hạng đầu của $g = \ln(f)$. Gọi hàm số học chỉ giữ $\sqrt{n}$ hạng đầu của g là g_0 , và đặt $g_1 = g - g_0$. Khi đó

$$\exp(g_0 + g_1) = \exp(g_0) \exp(g_1).$$

Chú ý rằng tích chập của hai g_1 có hạng thấp nhất $> n$, nên mọi giá trị trên sàng khối đều bằng 0 và không cần tính. Vì vậy

$$f = \exp(g_0) + \exp(g_0)g_1.$$

Sau khi tính $h = \exp(g_0)$, chỉ cần giải

$$f - h = hg_1.$$

Bây giờ ta cần giải một bài toán chia trên sàng khối. Với các hàm số học A, B, C thỏa $B(1) = 1$ và $A * B = C$, làm sao lấy sàng khối của A từ sàng khối của B, C ?

Trước hết, $\sqrt{n}$ hạng đầu của A dễ giải trong $O(\sqrt{n} \log n)$. Đặt $A = A_0 + A_1$, trong đó A_0 chỉ giữ giá trị của A tại các vị trí $\leq \sqrt{n}$, còn A_1 chỉ giữ giá trị tại các vị trí $> \sqrt{n}$. Khi đó

$$C = A_0B + A_1B.$$

Dùng một lần tích chập sàng khối để lấy A_0B , đặt $D = C - A_0B$; ta chỉ cần giải $A_1B = D$.

Với mọi k ,

$$D\left(\left\lfloor \frac{n}{k} \right\rfloor\right) = \sum_{i=1}^{\infty} B(i) S_{A_1}\left(\left\lfloor \frac{n}{ik} \right\rfloor\right).$$

Vì $S_{A_1}(\lfloor n/(ik) \rfloor)$ chỉ có giá trị khi $ik \leq \sqrt{n}$, tổng số i cần liệt kê là

$$\sum_{k=1}^{\sqrt{n}} \left\lfloor \frac{\sqrt{n}}{k} \right\rfloor = O(\sqrt{n} \log n).$$

Do đó thời gian là $O(\sqrt{n} \log n)$.

Tới đây, ta đã thực hiện được phép ln trên sàng khối bằng một lần exp của hàm chỉ có giá trị tại số nguyên tố, một lần tích chập sàng khối, và thêm $O(\sqrt{n} \log n)$ độ phức tạp phụ.

4.3. Tối ưu tích chập sàng khối

Lúc này nút thắt độ phức tạp lại nằm ở tích chập sàng khối. Tích chập sàng khối có thể làm tốt hơn không? Cũng có thể.

Trong phần sửa sai của ước lượng tích chập sàng khối, với mỗi $m \in [n - L, n]$, ta cần liệt kê mọi bộ ba (x, y, t) thỏa $xyt = m$. Cố định m , liệu phần này có thể tốt hơn $d_3(m)$?

Quan sát các giá trị ta thật sự quan tâm:

- giá trị của xy và t , vì chúng ảnh hưởng tới vị trí cần sửa;
- giá trị của $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor$, vì nó ảnh hưởng tới việc vị trí xy có được cập nhật hay không.

Chú ý tính chất

$$S \ln(xy) \leq \lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor < S \ln(xy) + 2.$$

Vì vậy khi đã xác định xy , giá trị $\lfloor S \ln x \rfloor + \lfloor S \ln y \rfloor$ chỉ có hai khả năng. Do đó với x, y , ta chỉ cần biết

$$w_x = \lfloor S \ln x \rfloor \bmod 2, \quad w_y = \lfloor S \ln y \rfloor \bmod 2.$$

Ta cần làm một phép chập để các đóng góp x, y đi vào xy . Giả sử phân tích thừa số nguyên tố của m là

$$m = \prod_{i=1}^k p_i^{\alpha_i}.$$

Phép chập ở trên thực chất là một phép chập k -chiều; miền giá trị của chiều thứ i là $[0, \alpha_i]$. Tuy phép chập còn chịu ảnh hưởng của w , chỉ cần liệt kê bốn trường hợp $w_x = 0/1$, $w_y = 0/1$ và làm phép chập nhiều chiều thông thường, hoặc dùng phản diễn đơn vị căn bậc hai: với $c = 1, -1$, dùng

$$c^{w_x} c^{w_y} = c^{(w_x + w_y) \bmod 2}$$

để giảm còn hai phép chập.

Bài toán chập đa thức nhiều biến có thể tham khảo luận văn đội tuyển tập huấn năm 2021 của Li Baitian [2]. Thời gian một lần là

$$d(m) \omega(m) \log d(m).$$

Xét cách phân tích độ phức tạp. Chọn một hằng số l . Số thừa số nguyên tố $> n^{1/l}$ chỉ là $O(l)$, và đóng góp của chúng vào $\omega(m)$ cũng như $\log d(m)$ đều không quá hằng số, nên có thể bỏ qua.

Với hạng $\log d(m)$, có thể dùng

$$\sum_{i=1}^k \sum_{e=1}^{\alpha_i} \frac{1}{e}$$

để ước lượng. Ngoài ra, nếu $p^t \parallel m$, thì

$$(t+1)d\left(\frac{m}{p^t}\right) \leq d(m),$$

nên có thể ước lượng $d(ipq^e)$ với $p, q \in \mathbb{P}$ bởi $O(d(i)e)$.

Với một m , phần $d(m) \log d(m) \omega(m)$ có thể viết ở cấp

$$O\left(d(m) \left(\sum_{\substack{p \mid m \\ p \in \mathbb{P}}} 1\right) \left(\sum_{\substack{q \in \mathbb{P}, e \geq 1 \\ q^e \parallel m}} \frac{1}{e}\right)\right).$$

Trường hợp $p = q$ có thể bỏ qua trong phân tích độ phức tạp. Ta có thể ước lượng tiếp bởi

$$O\left(\sum_{p|m, p \in \mathbb{P}} \sum_{\substack{q \in \mathbb{P}, e \geq 1 \\ q^e \parallel m, p \neq q}} e \cdot d\left(\frac{m}{pq^e}\right)\right).$$

Do đó có thể phân tích bằng tổng

$$\begin{aligned} & \sum_{m \in [n-L, n]} \sum_{\substack{p < n^{1/l} \\ p \in \mathbb{P}, p|m}} \sum_{\substack{q < n^{1/l} \\ q \in \mathbb{P}, e \geq 1, p \neq q}} d\left(\frac{m}{pq^e}\right) \\ & \leq \sum_{\substack{p \leq n^{1/l} \\ p \in \mathbb{P}}} \sum_{\substack{q \leq n^{1/l} \\ q \in \mathbb{P}, e \geq 1}} \sum_{i \in \left[\frac{n-L}{pq^e}, \frac{n}{pq^e}\right]} d(i). \end{aligned}$$

Chú ý rằng p, q, e chỉ có $O(n^{2/l} / \ln n)$ khả năng, nên tiếp tục dùng ước lượng tổng tiền tố hàm ước trong số học giải tích [1]:

$$\sum_{i \in [L, R]} d(i) = (R - L + 1) \log R + O(R^{131/416}).$$

Chỉ cần hằng số l thỏa

$$\frac{2}{l} + \frac{131}{416} < \frac{1}{2},$$

ta có thể bỏ qua $O(R^{131/416})$.

Vì vậy có thể ước lượng độ phức tạp tiếp thành

$$\sum_{\substack{p \leq n^{1/l} \\ p \in \mathbb{P}}} \sum_{\substack{q \leq n^{1/l} \\ q \in \mathbb{P}, e \geq 1}} \frac{L}{pq^e}.$$

Do

$$\sum_{\substack{p \leq S \\ p \in \mathbb{P}}} \frac{1}{p} = O(\log \log S),$$

và

$$\sum_{\substack{p \leq S \\ p \in \mathbb{P}, e \geq 1}} \frac{1}{p^e}$$

không vượt quá hai lần tổng trước, phần độ phức tạp này ước lượng được là

$$L(\log \log n)^2.$$

Tổng độ phức tạp là

$$\frac{n \log^2 n}{S} + S(\log \log n)^2.$$

Khi lấy

$$S = \frac{\sqrt{n} \log n}{\log \log n},$$

độ phức tạp thời gian là

$$O(\sqrt{n} \log n \log \log n).$$

5. Tổng kết

Trong bài viết này, tác giả thu được một phương pháp sàng có tính mở rộng khá tốt, đưa tích chập sàng khối, phép chia sàng khối, tổng tiền tố sàng khối của hàm nhân tính và bài toán thống kê tiền tố số nguyên tố về độ phức tạp thời gian $O(\sqrt{n} \text{ polylog } n)$. Hai bài toán sau cần thỏa một số điều kiện, chẳng hạn $f(p)$ là đa thức bậc hằng theo p , hoặc một vài tính chất đặc biệt khác. Bài viết cũng thảo luận tối ưu phương pháp này, đưa tích chập sàng khối và phép chia sàng khối của hàm số học về

$$O(\sqrt{n} \log n \log \log n),$$

đồng thời đưa tổng tiền tố sàng khối của hàm nhân tính và thống kê tiền tố số nguyên tố về

$$O\left(\frac{\sqrt{n} \log^2 n}{\sqrt{\log \log n}}\right).$$

Tuy độ phức tạp thời gian tốt hơn, do hằng số lớn và có nhiều thừa số log, cách làm này trong phạm vi dữ liệu OI hiện nay vẫn không thể vượt qua sàng Min_25. Tác giả hy vọng người đọc tiếp tục suy nghĩ và nghiên cứu bài toán tính tổng hàm nhân tính, và mong sẽ xuất hiện các cách làm thực dụng hơn.

6. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các huấn luyện viên đội tuyển tập huấn quốc gia, Peng Sijin và Yang Yaoliang, đã hướng dẫn.

Cảm ơn các thầy ở Trường Trung học Học Quân, trong đó có Xu Xianyou, đã quan tâm và hướng dẫn.

Cảm ơn gia đình đã đồng hành và ủng hộ.

Cảm ơn các bạn học và anh chị trong đội bạn đã giúp đỡ và đồng hành.

Cảm ơn các bạn học Zhang Yongxuan, Zhang Hengyi và những người khác đã đọc bản thảo và góp ý cho bài viết này.

Cảm ơn các thầy cô và bạn học khác đã giúp đỡ tác giả.

Tài liệu tham khảo

1. [Divisor summatory function](#).
2. Li Baitian. *Khung lý thuyết tính toán hàm sinh trong thi tin học*. Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2021, 2021.

Bài 2. Bàn về bài toán parity của matroid tuyến tính

Guo Yuchong, Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

Tóm tắt

Bài toán parity của matroid tuyến tính (*Linear Matroid Parity Problem*, LMPP) định nghĩa một lớp bài toán tối ưu hóa trên không gian tuyến tính. Mô hình này tạo cầu nối giữa phương pháp đại số và tối ưu tổ hợp: nhiều bài toán kinh điển như matching cực đại trên đồ thị tổng quát hoặc luồng mạng đều có thể quy về nó. Đổi lại, độ phức tạp thường tăng lên, nhưng ta thu được một góc nhìn thống nhất cho các bài toán bề ngoài rất khác nhau.

1. Ký hiệu và quy ước

Ký hiệu e_i là vector đơn vị có thành phần thứ i bằng 1, các thành phần còn lại bằng 0. Với ma trận A và hai tập chỉ số S, T , ký hiệu $A_{S,T}$ là ma trận con chỉ giữ các hàng trong S và các cột trong T . Ký hiệu $\deg_x(F)$ là bậc cao nhất theo biến x trong đa thức nhiều biến F .

2. Bài toán parity của matroid tuyến tính

2.1. Định nghĩa

Cho m cặp vector (a_i, b_i) , trong đó a_i, b_i đều là vector n -chiều. Cần chọn nhiều cặp nhất sao cho mọi vector xuất hiện trong các cặp được chọn đều độc lập tuyến tính.

2.2. Dạng thừa và dạng gọn

Gọi v là đáp án của bài toán gốc. Với các biến chưa xác định $x'_1, \dots, x'_m$, dựng

$$A = (a_1, b_1, a_2, b_2, \dots, a_m, b_m),$$

và

$$X = \text{diag} \left(\begin{pmatrix} 0 & x'_1 \\ -x'_1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & x'_2 \\ -x'_2 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & x'_m \\ -x'_m & 0 \end{pmatrix} \right).$$

Xét các ma trận trên trường phân thức hữu tỉ của $x'_1, \dots, x'_m$, ta có

$$2v + 2m = \text{rank} \begin{pmatrix} 0 & A \\ -A^T & X \end{pmatrix}.$$

Đây là cách dựng dạng thừa.

Với các biến chưa xác định $x_1, \dots, x_m$, dựng ma trận

$$M = \sum_{i=1}^m x_i (a_i b_i^T - b_i a_i^T).$$

Khi xét trên trường phân thức hữu tỉ của $x_1, \dots, x_m$, ta có

$$2v = \text{rank}(M).$$

Đây là cách dựng dạng gọn. So với dạng thừa, dạng gọn cho ma trận nhỏ hơn nên có lợi về thời gian tính toán; các thuật toán phía sau đều dùng cách dựng này. Tính đúng của hai cách dựng đã được chứng minh chi tiết trong tài liệu tham khảo [1], nên bài viết không lặp lại.

2.3. Thuật toán

Để có độ phức tạp tốt, ta xử lý đặc biệt các biến $x_1, \dots, x_m$. Chọn một số nguyên tố lớn p , lấy ngẫu nhiên các x_i trong $[0, p) \cap \mathbb{Z}$, rồi thế vào M để được ma trận M' trên $\mathbb{F}_p$. Dùng khử Gauss tính $\text{rank}(M')$ trong $O(n^3)$, và coi

$$\text{rank}(M) = \text{rank}(M')$$

để suy ra đáp án.

2.4. Phân tích xác suất đúng

Thuật toán trên có xác suất sai. Hiển nhiên $\text{rank}(M') \leq \text{rank}(M)$; cần chứng minh hai giá trị này bằng nhau với xác suất lớn.

Đặt $r = \text{rank}(M)$. Khi đó M có một ma trận con khả nghịch cấp r , ký hiệu $M_{S,T}$. Định thức $\det(M_{S,T})$ là một đa thức nhiều biến khác 0 theo $x_1, \dots, x_m$, còn $\det(M'_{S,T})$ là giá trị sau khi thế các biến ngẫu nhiên vào đa thức ấy.

Bổ đề Schwartz–Zippel. Nếu $f(x_1, \dots, x_m)$ là đa thức nhiều biến khác 0 có tổng bậc không quá d , và mỗi x_i được chọn ngẫu nhiên từ tập hữu hạn S , thì

$$\Pr[f(x_1, \dots, x_m) = 0] \leq \frac{d}{|S|}.$$

Chứng minh. Với $m = 1$, đa thức một biến bậc không quá d có không quá d nghiệm. Giả sử mệnh đề đúng với $m - 1$ biến, tách riêng biến x_m :

$$f(x_1, \dots, x_m) = \sum_{i=0}^d x_m^i f_i(x_1, \dots, x_{m-1}).$$

Gọi k là bậc cao nhất theo x_m . Sau khi cố định $x_1, \dots, x_{m-1}$, nếu đa thức một biến thu được là đa thức không thì

$$f_k(x_1, \dots, x_{m-1}) = 0,$$

mà f_k có bậc không quá $d - k$, nên xác suất trường hợp này không vượt quá $(d - k)/|S|$. Ngược lại, đa thức một biến không không có quá k nghiệm, nên xác suất chọn phải nghiệm không vượt quá $k/|S|$. Tổng lại là $d/|S|$.

Áp dụng bổ đề cho $\det(M_{S,T})$, xác suất $\det(M'_{S,T}) = 0$ không vượt quá r/p . Vì thế thuật toán cho đáp án đúng với xác suất ít nhất $1 - r/p$.

2.5. Dựng phương án

Xét lần lượt từng cặp vector. Nếu xóa cặp hiện tại mà đáp án không giảm thì xóa, ngược lại giữ lại. Cuối cùng sẽ còn đúng v cặp, tạo thành một phương án tối ưu. Cách làm trực tiếp cần tính lại $\text{rank}(M)$ sau mỗi lần xóa, nên mất $O(mn^3)$.

Vì

$$\text{rank}(a_i b_i^T - b_i a_i^T) \leq 2,$$

có thể tăng tốc bằng kỹ thuật nhiều hạng thấp của ma trận.

Bổ đề Sherman–Morrison–Woodbury. Với ma trận khả nghịch A, B , ma trận U cỡ $n \times m$ và ma trận V cỡ $m \times n$, ta có

$$(A - UB^{-1}V)^{-1} = A^{-1} + A^{-1}U(B - VA^{-1}U)^{-1}VA^{-1}.$$

Đồng thời $A - UB^{-1}V$ khả nghịch khi và chỉ khi $B - VA^{-1}U$ khả nghịch.

Trong bài toán hiện tại, ma trận M dựng được chưa chắc khả nghịch nên chưa thể dùng trực tiếp bổ đề. Do M phản đối xứng, nếu $r = \text{rank}(M)$ thì M có một ma trận con chính $M_{S,S}$ khả nghịch cấp r . Chỉ giữ các thành phần vector thuộc S sẽ đưa bài toán về trường hợp M khả nghịch mà không làm mất phương án đúng; một tập S hợp lệ có thể tìm trong $O(n^3)$.

Giả sử từ đây M khả nghịch và duy trì $N = M^{-1}$. Với cặp đang xét, đặt

$$U = (a_i, -b_i), \quad V = (b_i, a_i)^T,$$

khi đó $a_i b_i^T - b_i a_i^T = UV$. Việc kiểm tra $M - UV$ khả nghịch được chuyển thành kiểm tra ma trận 2×2

$$I - VNU.$$

Ta tính nó trong $O(r^2)$ và kiểm tra định thức trong $O(1)$. Nếu khả nghịch, cập nhật

$$N' = (M - UV)^{-1} = N + NU(I - VNU)^{-1}VN,$$

cũng trong $O(r^2)$. Tổng thời gian dựng phương án giảm còn $O(mr^2)$, sau phần tiền xử lý $O(n^3)$.

2.6. Mở rộng

Định nghĩa LMPP yêu cầu mỗi nhóm có đúng hai vector. Với trường hợp có nhóm một vector, cách trực tiếp là mở rộng vector lên $n + m_1$ chiều, và với nhóm một vector thứ i ghép thêm $b_i = e_{n+i}$. Cách này đúng nhưng làm tăng số chiều khi m_1 lớn.

Nếu viết ma trận sau mở rộng dưới dạng

$$M = \begin{pmatrix} M_2 & A_1 \\ -A_1^T & 0 \end{pmatrix},$$

trong đó M_2 là ma trận chỉ xét các cặp và A_1 chứa các vector của nhóm một phần tử, thì ta chỉ quan tâm tới $\text{rank}(M)$. Vì vậy có thể chỉ giữ một cơ sở cột của A_1 , kích thước không quá n , rồi quy về trường hợp $m_1 \leq n$. Khi ấy cách mở rộng trực tiếp không làm tăng cấp độ phức tạp.

2.7. Tiểu kết

Phần này đã trình bày thuật toán ngẫu nhiên tổng quát để giải và dựng phương án cho LMPP, đồng thời nêu cách mở rộng sang nhóm một vector mà không tăng cấp độ phức tạp. Các thành phần thời gian chính là:

- dựng $M = \sum_{i=1}^m x_i(a_i b_i^T - b_i a_i^T)$, mất $O(mn^2)$;
- tính $\text{rank}(M)$, mất $O(n^3)$;
- nếu cần dựng phương án, tính M^{-1} , mất $O(n^3)$;
- sau mỗi nhiễu hạng thấp, kiểm tra khả nghịch và cập nhật N , tổng $O(mr^2)$.

Trong bài toán cụ thể, cấu trúc đặc biệt của a_i, b_i thường còn cho phép tối ưu thêm.

3. Bài toán parity có trọng số của matroid tuyến tính

3.1. Giới thiệu

Bài toán parity có trọng số là phiên bản có trọng số của LMPP. Tài liệu tham khảo [4] đưa ra thuật toán $O(\text{poly}(n, m))$ cho bài toán này, nhưng thuật toán khá phức tạp. Phần này trình bày một thuật toán đơn giản hơn với thời gian cao hơn, không phải đa thức theo độ dài biểu diễn tổng quát.

3.2. Định nghĩa

Cho m cặp vector (a_i, b_i) , trong đó a_i, b_i là vector n -chiều. Cặp thứ i có trọng số nguyên dương w_i . Cần chọn một số cặp có tổng trọng số lớn nhất sao cho mọi vector trong các cặp được chọn độc lập tuyến tính.

3.3. Dạng thừa và dạng gọn

Gọi v là đáp án, và giả sử $w_i \in [1, V] \cap \mathbb{Z}$. Thêm biến chưa xác định z , dùng bậc theo z để biểu diễn tổng trọng số. Tương tự dạng thừa không trọng số, đặt

$$A = (a_1 z^{w_1}, b_1, a_2 z^{w_2}, b_2, \dots, a_m z^{w_m}, b_m),$$

$$X = \text{diag} \left(\begin{pmatrix} 0 & x'_1 \\ -x'_1 & 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 & x'_m \\ -x'_m & 0 \end{pmatrix} \right), \quad Y = \text{diag}(y_1, \dots, y_n).$$

Khi đó

$$2v = \deg_z \det \begin{pmatrix} Y & A \\ -A^T & X \end{pmatrix}.$$

Ý tưởng chứng minh là khai triển định thức theo các phần tử đường chéo của Y , rồi dùng tính chất của Pfaffian:

$$\det(B) = \text{pf}(B)^2$$

với B phản đối xứng. Các tiểu thức khác 0 của X buộc hai chỉ số của mỗi cặp phải cùng được chọn hoặc cùng không được chọn; bậc cao nhất theo z khi đó đúng bằng hai lần tổng trọng số lớn nhất của một tập vector độc lập tuyến tính.

Để đưa về dạng gọn, dùng bổ đề Schur. Nếu D khả nghịch thì

$$\det \begin{pmatrix} A & B \\ C & D \end{pmatrix} = \det(A - BD^{-1}C) \det(D).$$

Áp dụng vào ma trận trên:

$$2v = \deg_z \det(Y + AX^{-1}A^T).$$

Đặt $x_i = -1/x'_i$, ta có

$$Y + AX^{-1}A^T = Y + \sum_{i=1}^m x_i (a_i b_i^T - b_i a_i^T) z^{w_i}.$$

Gọi ma trận này là M , ta thu được dạng gọn của bài toán có trọng số:

$$2v = \deg_z \det(M).$$

3.4. Thuật toán

Chỉ cần tính $\det(M)$. Tương tự trường hợp không trọng số, các biến x_i, y_i được gán ngẫu nhiên trên $\mathbb{F}_p$. Mỗi phần tử của M là đa thức theo z bậc không quá V , nên $\det(M)$ có bậc không quá Vn . Chọn $Vn + 1$ giá trị khác nhau $z_1, \dots, z_{Vn+1}$, tính $\det(M)$ sau khi thế từng z_i , rồi dùng nội suy Lagrange để khôi phục đa thức $\det(M)$.

3.5. Dựng phương án

Tương tự phần không trọng số, duy trì các ma trận sau khi thế $z_1, \dots, z_{V_{n+1}}$:

$$M_1, \dots, M_{V_{n+1}}$$

cùng nghịch đảo của chúng

$$N_1, \dots, N_{V_{n+1}}.$$

Ta cần hỗ trợ nhiều hạng thấp và duy trì định thức.

Bổ đề định thức ma trận. Nếu A, B khả nghịch thì

$$\det(A - UB^{-1}V) = \frac{\det(A)}{\det(B)} \det(B - VA^{-1}U).$$

Chúng minh tương tự bổ đề Sherman–Morrison–Woodbury.

Nếu mọi M_i luôn khả nghịch thì công thức trên đủ để cập nhật định thức. Trong thực tế có thể gặp thời điểm $\det(M_i) = 0$. Vì $\det(M)$ có bậc không quá Vn theo z , nếu các z_i được chọn ngẫu nhiên trong $[0, p) \cap \mathbb{Z}$, thì theo Schwartz–Zippel, số lần kỳ vọng gặp $\det(M_i) = 0$ là

$$O\left(\frac{V^2 mn^2}{p}\right).$$

Khi xảy ra, chọn lại z_i ngẫu nhiên cho tới khi nó khác mọi z_j còn lại và $\det(M_i) \neq 0$. Với p đủ lớn, chi phí dựng lại có thể xem là không đáng kể. Sau đó, mỗi lần nội suy lại $\det(M)$ từ các giá trị $\det(M_1), \dots, \det(M_{V_{n+1}})$, ta quyết định được cặp hiện tại có nằm trong phương án tối ưu hay không.

3.6. Tiểu kết

Phần này trình bày thuật toán ngẫu nhiên tổng quát cho phiên bản có trọng số. Các thành phần thời gian chính là:

- tính giá trị của từng phần tử ma trận tại $z_1, \dots, z_{V_{n+1}}$ bằng đánh giá nhiều điểm: $O(Vn^3 \log^2(Vn))$;
- tính định thức tại từng z_i : $O(Vn^4)$;
- nội suy $\det(M)$: $O(Vn \log^2(Vn))$.

Nếu cần dựng phương án, cần thêm:

- tính $N_1, \dots, N_{V_{n+1}}$: $O(Vn^4)$;
- sau mỗi nhiều hạng thấp, nội suy lại $\det(M)$: $O(Vmn \log^2(Vn))$;
- cập nhật các $\det(M_i)$ và N_i : $O(Vmn^3)$.

4. Ứng dụng của bài toán parity matroid tuyến tính

4.1. Giới thiệu

Phần này nêu một số ví dụ quy các bài toán tối ưu tổ hợp về LMPP.

4.2. Matching cực đại trên đồ thị tổng quát

Định nghĩa. Cho một đồ thị vô hướng, cần chọn nhiều cạnh nhất sao cho không có hai cạnh được chọn chung đỉnh.

Thuật toán. Cách cổ điển để giải bài toán này là thuật toán blossom. Với cạnh thứ i là (u_i, v_i) , đặt

$$a_i = e_{u_i}, \quad b_i = e_{v_i}.$$

Nếu hai cạnh được chọn chung đỉnh u , vector e_u xuất hiện ít nhất hai lần nên phụ thuộc tuyến tính. Ngược lại, nếu không chung đỉnh thì mỗi vector đơn vị xuất hiện nhiều nhất một lần, nên độc lập tuyến tính. Do đó bài toán matching cực đại được quy về LMPP. Ma trận M thu được trùng với ma trận Tutte, một công cụ kinh điển khác cho matching tổng quát.

Nếu số cạnh là m , số đỉnh là n , dùng trực tiếp thuật toán tổng quát để dựng phương án cho thời gian $O(mn^2)$.

Tối ưu. Ta dùng cấu trúc đặc biệt của a_i, b_i . Bổ đề Harvey nói rằng nếu A khả nghịch và A, B chỉ khác nhau ở ma trận con chính ứng với tập S , đặt $D = B - A$, thì

$$(B^{-1})_{T,T} = (A^{-1})_{T,T} - (A^{-1})_{T,S}(I + D_{S,S}(A^{-1})_{S,S})^{-1}D_{S,S}(A^{-1})_{S,T}.$$

Đồng thời B khả nghịch khi và chỉ khi

$$I + D_{S,S}(A^{-1})_{S,S}$$

khả nghịch. Bổ đề này cho thấy chỉ cần thông tin cục bộ để xử lý một sửa đổi cục bộ.

Trong matching tổng quát, cặp (a_i, b_i) chỉ ảnh hưởng hai vị trí M_{u_i, v_i} và M_{v_i, u_i} . Vì vậy có thể dùng chia để trị. Gọi $\text{solve}(S, T)$ là thủ tục xử lý mọi cặp có $u_i \in S, v_i \in T$, quyết định chúng có thuộc đáp án hay không. Đặt

$$X = S \cup T,$$

ta chỉ cần quan tâm $M_{X,X}$ và $(M^{-1})_{X,X}$. Nếu $|S| = |T| = 1$, xử lý trực tiếp trong $O(1)$. Ngược lại, chia đôi S, T thành S_1, S_2, T_1, T_2 , đệ quy trên bốn cặp (S_i, T_j) ; sau mỗi đệ quy, các cặp bị loại chỉ gây thay đổi trong ma trận con chính ứng với X , nên dùng bổ đề Harvey cập nhật ảnh hưởng lên $(M^{-1})_{X,X}$ trong $O(|X|^3)$.

Phân tích đệ quy cho hai trường hợp $S = T$ và $S \cap T = \emptyset$ dẫn tới

$$T_2(n) = O(n^3) + 4T_2(n/2) = O(n^3),$$

$$T_1(n) = 2T_1(n/2) + 2T_2(n) + O(n^3) = O(n^3).$$

Vì thế $\text{solve}(\{1, \dots, n\}, \{1, \dots, n\})$ chạy trong $O(n^3)$.

4.3. Cây khung có mex lớn nhất

Bài toán này xuất phát từ QOJ 6402, *MEXimum Spanning Tree*.

Định nghĩa. Cho đồ thị vô hướng liên thông có trọng số cạnh là số nguyên không âm. Cần tìm cây khung sao cho mex của tập trọng số cạnh là lớn nhất. Với tập S , $\text{mex}(S)$ là số nguyên không âm nhỏ nhất không thuộc S .

Thuật toán. Diễn đạt lại: cần tìm k lớn nhất sao cho có thể chọn một đồ thị con không chu trình, trong đó với mỗi trọng số thuộc $[0, k) \cap \mathbb{Z}$ có đúng một cạnh được chọn. Khi cố định k , đây là giao của matroid đồ thị (các rừng) và matroid màu (các tập có màu đôi một khác nhau); cả hai đều là matroid tuyến tính, và giao matroid tuyến tính yếu hơn LMPP.

Cụ thể, nếu số đỉnh là n , với cạnh (u_i, v_i, w_i) thỏa $w_i \in [0, k) \cap \mathbb{Z}$, đặt

$$a_i = e_{u_i} - e_{v_i}, \quad b_i = e_{n+w_i}.$$

Cách dựng này cũng cho thấy quy nạp tổng quát từ giao hai matroid tuyến tính về LMPP. Ma trận tương ứng có dạng

$$M = \begin{pmatrix} 0 & B \\ -B^T & 0 \end{pmatrix},$$

nên $\text{rank}(M) = 2 \text{rank}(B)$.

Trong bài toán này, duyệt k tăng dần cho tới khi $\text{rank}(B) < k$. Khi chuyển từ $k - 1$ sang k , cột thứ k của B đổi từ toàn 0 thành

$$\sum_{i=1}^m [w_i = k] a_i.$$

Giống duy trì cơ sở tuyến tính, vừa thêm vector vừa khử Gauss; mỗi vector mất $O(n^2)$, tổng thời gian $O(n^3)$.

4.4. Cây khung với giới hạn số cạnh mỗi màu

Định nghĩa. Cho đồ thị vô hướng liên thông, mỗi cạnh có một màu là số nguyên dương. Cần tìm một rừng có nhiều cạnh nhất sao cho với mọi màu i , số cạnh màu i không vượt quá lim_i .

Thuật toán. Xem bài toán như giao matroid tuyến tính. Điểm chính là biểu diễn điều kiện “màu i xuất hiện không quá lim_i lần” bằng một matroid tuyến tính. Với màu i , cấp lim_i chiều độc lập với các màu khác. Bài toán quy về việc dựng m vector $k = \text{lim}_i$ chiều trên $\mathbb{F}_p$ sao cho bất kỳ k vector nào cũng độc lập tuyến tính.

Chọn một số nguyên tố $p > m$, và đặt thành phần thứ j của vector thứ i là i^j . Tính độc lập của mọi k vector được chứng minh trực tiếp bằng định thức Vandermonde.

4.5. Ví dụ khác

Bài toán này xuất phát từ UOJ 785, *Goodbye Renyin E: Tìm cặp năm mới*.

Định nghĩa. Cho đồ thị vô hướng, đỉnh u có màu w_u . Cần chọn nhiều đường đi đôi một không giao nhau nhất. Với mỗi đường đi P , gọi hai đầu mút là u, v , cần thỏa:

- $w_u \neq 0, w_v \neq 0$, và $w_u \neq w_v$;
- với mọi $x \in P, x \neq u, x \neq v$, ta có $w_x = 0$.

Thuật toán. Đổi cách nhìn: chọn nhiều cạnh nhất sao cho trong đồ thị chỉ giữ các cạnh ấy, mỗi thành phần liên thông có nhiều nhất hai đỉnh màu khác 0; nếu có hai đỉnh như vậy thì màu của chúng phải khác nhau. Chuyển đổi này đúng vì nghiệm tối ưu không có thành phần toàn màu 0 trừ trường hợp mọi đỉnh

đều màu 0; nếu có thể gộp thành phần ấy vào thành phần kề thì nghiệm tốt hơn. Nếu gọi c_0 là số đỉnh màu khác 0, và c_2 là số thành phần có đúng hai đỉnh màu khác 0, thì

$$|V| - \#E_{\text{chọn}} = \#\text{thành phần liên thông} = c_0 - c_2.$$

Hai đại lượng đầu và số đỉnh màu khác 0 là hằng số, nên chỉ cần tối đa hóa số cạnh.

Gán cho mỗi màu i một vector hai chiều a_i , sao cho các vector này đôi một độc lập tuyến tính. Với cạnh (u_i, v_i) , đặt

$$a_i = e_{2u_i-1} - e_{2v_i-1}, \quad b_i = e_{2u_i} - e_{2v_i}.$$

Cách này bảo đảm tập cạnh được chọn tạo thành rừng. Với đỉnh u có $w_u \neq 0$, đặt

$$a_u = \alpha_{w_u,1} e_{2u-1} + \alpha_{w_u,2} e_{2u}.$$

Sau đó có hai cách tương đương:

- xem mỗi đỉnh màu khác 0 là một nhóm một vector có thể chọn hoặc không chọn, rồi dùng dạng mở rộng ở phần 2.6;
- bắt buộc chọn mọi đỉnh màu khác 0. Nếu V_1 là không gian sinh bởi các vector của đỉnh, phân rã $\mathbb{F}_p^n = V_1 \oplus V_2$ và chiếu mọi a_i, b_i của cạnh xuống V_2 , từ đó loại ảnh hưởng của điều kiện bắt buộc chọn đỉnh.

Trong nghiệm tối ưu, mọi đỉnh màu khác 0 đều được chọn, nên hai cách là tương đương. Để kiểm tra rằng cấu trúc trên bảo đảm mỗi thành phần liên thông thỏa yêu cầu. Do a_i, b_i có dạng giống ví dụ matching, có thể dùng tiếp kỹ thuật chia để trị dựa trên bổ đề Harvey để đạt tổng thời gian $O(n^3)$.

5. Tổng kết

Bài viết giới thiệu thuật toán tổng quát cho bài toán parity của matroid tuyến tính, các mở rộng và một số ứng dụng; qua đó khảo sát bước đầu việc dùng phương pháp đại số trong tối ưu tổ hợp. Do giới hạn năng lực của tác giả, bài viết không đi vào các nội dung như thuật toán đa thức cho phiên bản có trọng số hoặc bài toán parity của matroid phi tuyến tính.

Những năm gần đây, học thuật có nhiều kết quả mới theo hướng này, đồng thời còn nhiều phương pháp đại số khác đang phát triển. Hiện tại các phương pháp ấy chưa được cộng đồng OI tiếp nhận rộng rãi. Tác giả hy vọng bài viết có thể gợi hứng thú tìm hiểu phương pháp đại số và đưa những lớp bài toán, kỹ thuật mới vào OI.

6. Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Jin Jing, Gao Kai và Wu Shenguang đã hướng dẫn tác giả.

Cảm ơn cha mẹ đã nuôi dưỡng và dạy dỗ.

Cảm ơn các anh Chang Ruinian, Wan Chengzhang, cùng các bạn Ke Yisi và Ke Yijing đã giúp đỡ trong quá trình viết bài.

Cảm ơn mọi thầy cô và bạn học đã giúp đỡ tác giả.

Tài liệu tham khảo

1. Chang Ruinian. *Bàn về ứng dụng đại số tuyến tính trong OI*. Tập luận văn đội tuyển quốc gia IOI 2022.

2. Ren Qingyu. *Goodbye Renyin E: lời giải bài Tìm cặp năm mới*.
3. Ho Yee Cheung, Lap Chi Lau, Kai Man Leung. *Algebraic Algorithms for Linear Matroid Parity Problems*.
4. Satoru Iwata, Yusuke Kobayashi. *A Weighted Linear Matroid Parity Algorithm*.

Bài 3. Bàn về hợp nhất và tách cấu trúc dữ liệu

Huang Luotian, Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc

Tóm tắt

Bài viết thảo luận việc hợp nhất và tách các cấu trúc dữ liệu. Trước hết ta giới thiệu hợp nhất và tách cây đoạn, cây cân bằng, rồi mở rộng sang cây phân trị đỉnh, cây phân trị cạnh và static Top Tree; đồng thời đưa ra cách hợp nhất và tách static Top Tree. Ngoài ra, bài viết cũng trình bày một số ứng dụng của những kỹ thuật này trong OI.

1. Hợp nhất và tách cấu trúc dữ liệu tĩnh

1.1. Dẫn nhập

Các cấu trúc dữ liệu tĩnh quen thuộc có nhiều loại: cây đoạn, cây phân trị đỉnh, cây phân trị cạnh, static Top Tree. Do đặc điểm tĩnh của chúng, khi cần hợp nhất hoặc tách, thao tác thường khá thuận tiện: thông tin tại các nút tương ứng có thể được hợp nhất trực tiếp; khi tách, chỉ cần tách một số nút thành hai nút nằm ở cùng vị trí. Điều này cũng thể hiện rõ trong hợp nhất cây đoạn.

1.2. Hợp nhất cây đoạn

Trước hết xét hợp nhất cây đoạn không có lazy tag, quan sát thuật toán cơ bản rồi thử mở rộng.

Hợp nhất cây đoạn nghĩa là hợp nhất hai cây đoạn mở nút động thành một cây, đồng thời hợp nhất thông tin tại các vị trí tương ứng. Ý tưởng thô bạo là đệ quy hợp nhất mọi nút xuất hiện đồng thời trong hai cây; nếu một bên là nút rỗng và bên kia là một cây con, thì trả thẳng cây con ấy.

Để duy trì thông tin, với nút được hợp nhất thô bạo ta có hai lựa chọn:

1. sau khi hợp nhất xong các cây con, kéo thông tin từ hai con mới lên;
2. hợp nhất trực tiếp thông tin của hai nút tương ứng.

Hai cách này có ưu và nhược điểm riêng. Cách thứ nhất đòi hỏi thông tin trên cây đoạn hỗ trợ hợp nhất từ các con. Trong đa số tình huống đây là điều tự nhiên; ví dụ với dãy nhị phân, ta có thể duy trì trong mỗi đoạn số lần xuất hiện của các dãy con $[010, 01, 10, 0, 1]$.

Nhưng có những thông tin không hỗ trợ kéo từ con lên. Chẳng hạn ta hợp nhất cây đoạn lồng cây cân bằng, tức mỗi nút cây đoạn duy trì một cây cân bằng. Nếu hợp nhất cây cân bằng từ hai con rồi cần sao chép một bản làm thông tin của nút hiện tại, không phải mọi loại cây cân bằng hoặc thao tác đều có thể được làm bền vững. Trong khi đó, hợp nhất theo vị trí các cây cân bằng có thể xem như thực hiện đồng thời nhiều phép hợp nhất cây cân bằng, nên có thể dùng lại phân tích độ phức tạp của hợp nhất cây cân bằng.

Trong phạm vi xử lý được bởi hai cách trên, ta có thể hợp nhất hai cây trong thời gian

$$O(|T_1 \cap T_2|).$$

Sau n lần sửa đổi và hợp nhất, tổng độ phức tạp thời gian là $O(n \log n)$.

Với cây đoạn có lazy tag, do cây đoạn mở nút động có thể tạo nút mới khi đẩy tag xuống, ảnh hưởng tới độ phức tạp, ta cần dùng một số cách để hạn chế số nút mới.

Với cây đoạn mở nút động đang được duy trì, luôn giữ tính chất: mỗi nút hoặc không có con, hoặc có đủ hai con. Khi đó đẩy tag ở nút không phải lá sẽ không tạo nút mới. Bài toán trở thành: xử lý thế nào khi hợp nhất một nút lá với một cây con cây đoạn. Một ý tưởng là nút lá thường biểu diễn một đoạn có tính chất đồng nhất, nên trích ra tính chất đồng nhất đó và đánh thành tag lên cây con còn lại. Ví dụ nếu sửa đổi là cộng đoạn, thì mọi giá trị trong đoạn do nút lá biểu diễn đều giống nhau. Nếu hợp nhất hai cây là cộng theo vị trí, ta có thể xem giá trị đồng nhất ấy là một tag cộng cho cây con của cây kia.

Ví dụ 1.1. Cho một cây nhị phân gốc 1, mỗi đỉnh có trọng số a_i . Mỗi đỉnh hoặc là lá, hoặc có đúng hai con. Nếu đỉnh u là lá thì $b_u = a_u$; ngược lại

$$b_u = |b_{\text{lson}} - b_{\text{rson}}| + a_u,$$

trong đó lson, rson là hai con trái phải của u . Hỗ trợ sửa đổi điểm a_u , sau mỗi lần hỏi b_1 , với

$$n, q \leq 10^5, \quad 0 \leq a_u \leq 20.$$

Xét xử lý offline, dùng cây đoạn duy trì giá trị b_u của mỗi đỉnh u tại các thời điểm khác nhau. Với đỉnh không phải lá cần hợp nhất hai cây đoạn; với mọi đỉnh cần thực hiện một số phép cộng đoạn.

Vấn đề chính là xử lý nhanh việc hợp nhất một lá với một cây con cây đoạn mở nút động. Khi đó thao tác cần làm là biến mọi giá trị x trong cây con thành

$$x \mapsto |x - v|,$$

trong đó v là tag cộng đoạn trên nút lá. Rõ ràng lấy trị tuyệt đối theo v không phải một tag hợp nhất được, nên ta dùng phân tích thế năng.

Gọi $\min_u, \max_u$ lần lượt là giá trị nhỏ nhất và lớn nhất của b trong đoạn thời gian do nút cây đoạn u biểu diễn. Khi cần lấy trị tuyệt đối theo v trên cây con u , có hai nhánh cắt tĩa:

1. nếu $v \leq \min_u$, đánh tag $x \mapsto x - v$;
2. nếu $v \geq \max_u$, đánh tag $x \mapsto v - x$.

Vì phép $x \mapsto kx + b$ có thể hợp nhất, hai nhánh trên xử lý được bằng tag. Thiết kế thế năng

$$\Phi = \sum_u (\max_u - \min_u + 1).$$

Nếu không rơi vào hai nhánh cắt tĩa, đệ quy thô bạo sẽ làm $\max - \min$ giảm ít nhất 1. Phép cộng đoạn chỉ làm thay đổi thế năng của $O(\log n)$ đoạn, và giá trị v hiển nhiên thay đổi nhiều nhất chưa tới 20. Vì thế tổng độ phức tạp thời gian là $O(nv \log n)$. Ví dụ này minh họa một cách khác dùng thế năng khi hợp nhất nút lá và cây con cây đoạn.

1.3. Tối ưu độ phức tạp bộ nhớ

Nếu cài đặt trực tiếp, hợp nhất cây đoạn dùng $O(n \log n)$ bộ nhớ. Nếu các phép hợp nhất có thể xử lý offline, có thể tối ưu xuống $O(n)$.

Xét dựng cây từ quá trình hợp nhất, rồi phân rã nặng nhẹ trên cây hợp nhất này; chạy hợp nhất cây đoạn theo thứ tự ưu tiên đi vào con nặng, đồng thời viết thu hồi rác cho cây đoạn. Ta chứng minh bộ nhớ của cách này là $O(n)$.

Bổ đề 1.1. Một cây đoạn mở nút động sau m thao tác có

$$O(m(\log n - \log m))$$

nút.

Xét phân loại nút cây đoạn, lần lượt với $\text{dep} \leq \log m$ và $\text{dep} > \log m$. Với loại đầu, ngay cả khi xây đầy đủ cũng chỉ có $O(m)$ nút. Vì sửa đổi cây đoạn có dạng hai chuỗi, nên nhiều nhất có $O(m)$ chuỗi. Với loại sau, mỗi chuỗi chỉ có thể tạo mới $O(\log n - \log m)$ nút, nên tổng cộng có

$$O(m(\log n - \log m))$$

nút.

Bổ đề 1.2. Với một nút bất kỳ u , đường từ u tới gốc sẽ được chia thành $k \leq \log n$ tiền tố chuỗi nặng. Giả sử đỉnh đầu của các chuỗi nặng này, theo thứ tự từ trên xuống, là $z_1, z_2, \dots, z_k$, thì

$$\text{size}(z_i) \leq \frac{n}{2^{i-1}}.$$

Mỗi lần nhảy lên một chuỗi nặng, kích thước cây con ít nhất nhân đôi, suy ra bổ đề trên. Kết hợp hai bổ đề, tại mọi thời điểm tổng kích thước các cây đoạn bị chặn bởi

$$\sum_{i=1}^k O\left(\frac{n}{2^{i-1}} \left(\log n - \log \frac{n}{2^{i-1}}\right)\right) = \sum_{i=1}^k O\left(\frac{n}{2^i} i\right) = O(n).$$

Do đó với các bài toán ở phần sau có thể offline, bộ nhớ đều có thể tối ưu xuống $O(n)$.

Với một lớp cây đoạn mở nút động đặc biệt khác, mỗi lần sửa đổi đều là sửa điểm. Lợi dụng tính chất này cũng có thể tối ưu bộ nhớ xuống $O(n)$. Cụ thể, duy trì cây ảo của mọi nút lá trên cấu trúc cây đoạn và chỉ duy trì các nút thuộc cây ảo. Sau m thao tác, cây đoạn mở nút động hiển nhiên có $O(m)$ nút cây ảo. Tuy nhiên cách này khá rườm rà khi cài đặt, nên thông thường dùng phương pháp thứ nhất.

1.4. Hợp nhất thêm các cấu trúc dữ liệu tĩnh

1.4.1. Hợp nhất cây phân trị đỉnh, cây phân trị cạnh. Ta thấy hợp nhất cây đoạn có khả năng mở rộng rất mạnh.

Tương tự cây đoạn mở nút động, ta có thể định nghĩa cây phân trị đỉnh, cây phân trị cạnh mở nút động. Để tiện trình bày, ta chỉ bàn cây phân trị đỉnh; cây phân trị cạnh là tương tự.

Vì mỗi nút trong cây phân trị đỉnh có nhiều con, ta khó hỗ trợ tìm kiếm từ trên xuống, và một số thông tin cũng khó duy trì. Do đó xét xây cây Huffman cho các con của cây phân trị đỉnh; không khó chứng minh chiều cao là $O(\log n)$.

Trong một lớp bài toán cây phân trị đỉnh, ta thường cần duy trì thông tin từ trọng tâm tới các đỉnh khác trong thành phần liên thông hiện tại. Với mỗi đỉnh u , các nút bị nó ảnh hưởng chính là các tổ tiên của nó trên cây phân trị đỉnh. Vì vậy nếu ta làm sửa điểm thì chỉ có $O(\log n)$ nút có thông tin thay đổi trên cây phân trị đỉnh.

Khác cây đoạn, cây phân trị đỉnh không hỗ trợ hợp nhất thông tin duy trì ở các con lên nút hiện tại, nên chỉ có thể dùng cách hợp nhất thứ hai ở trên.

Ví dụ 1.2 (CTSC 2018, Viết sai bằng bạo lực). Cho hai cây có trọng số cạnh T, T' , trọng số cạnh có thể âm. Cần tìm hai đỉnh x, y để tối đa hóa

$$\text{depth}(x) + \text{depth}(y) - (\text{depth}(\text{LCA}(x, y)) + \text{depth}'(\text{LCA}'(x, y))).$$

Biến đổi phần đóng góp để loại LCA trong cây thứ nhất, thu được

$$\frac{1}{2}(\text{depth}(x) + \text{depth}(y) + \text{dis}(x, y) - 2 \text{depth}'(\text{LCA}'(x, y))).$$

Sau đó liệt kê $\text{LCA}'(x, y)$ trong T' , tức DFS trên T' . Giả sử liệt kê tới u ; mỗi lần hợp nhất một con v của u vào cây con của u và tính đóng góp.

Khi đó bài toán chuyển thành: ban đầu có nhiều tập, mỗi tập là một đỉnh; cần hỗ trợ hợp nhất tập. Khi hợp nhất S_1, S_2 , cần tính

$$\max_{u \in S_1, v \in S_2} \text{dis}(u, v) + \text{depth}(u) + \text{depth}(v).$$

Dùng cây phân trị cạnh để xử lý đóng góp. Với mỗi tập, mở một cây phân trị cạnh; mỗi nút lưu $\text{info}_{u,0/1}$, biểu thị giá trị lớn nhất của $\text{dep} + \text{dis}$ từ hai phía tới cạnh phân trị. Với mỗi tập, dùng cây phân trị cạnh mở nút động rồi hợp nhất các cây này. Vì thông tin của nút rỗng là đơn vị, nên hợp nhất nút rỗng với cây con thì trả thẳng cây con. Với hai nút trùng nhau, hợp nhất $\text{info}_{u,0/1}, \text{info}_{v,0/1}$ bằng phép lấy max. Tổng thời gian là $O(n \log n)$, bộ nhớ $O(n)$.

1.4.2. Hợp nhất static Top Tree. Trước hết ta giới thiệu static Top Tree.

Định nghĩa 1.1. Với cây vô hướng T , định nghĩa một *cluster* trên T là một bộ ba

$$C = (V, E, B),$$

trong đó (V, E) liên thông và là đồ thị con của T , tập $B \subseteq V, |B| \leq 2$, và với mọi $x \in V$, nếu tồn tại $y \notin V$ sao cho $(x, y) \in E(T)$, thì $x \in B$. Các phần tử trong B gọi là điểm biên của C . Ta dùng $V(C), E(C)$ lần lượt biểu thị tập đỉnh, tập cạnh của C , và $B(C)$ biểu thị tập điểm biên của C .

Định nghĩa 1.2. Thao tác *compress* là thao tác hợp nhất hai cluster thành một cluster. Trong đó, hai cluster có giao của hai tập điểm biên bằng 1 được hợp nhất thành một cluster mới; tập điểm biên mới là hiệu đối xứng của hai tập điểm biên cũ.

Định nghĩa 1.3. Thao tác *rake* cũng hợp nhất hai cluster thành một cluster. Trong đó, hai cluster có giao của hai tập điểm biên bằng 1 được hợp nhất thành một cluster mới; tập điểm biên mới là tập điểm biên của một trong hai cluster.

Với một cây, ta có thể dùng hai thao tác trên để co toàn bộ cây. Cụ thể, xem một cluster như một cạnh bị co giữa hai điểm biên (u, v) ; ban đầu mỗi cạnh (u, v) là một cluster, rồi liên tục hợp nhất các cluster bằng hai thao tác trên, cuối cùng hợp nhất được cả cây thành một cluster. Trong quá trình co, cấu trúc hợp nhất của các cluster tạo thành một cây, gọi là Top Tree.

Không khó thấy Top Tree như vậy có mỗi nút nhiều nhất hai con. Tiếp theo ta đưa ra một cách xây cây sao cho độ sâu của Top Tree là $\Theta(\log n)$.

Phân rã nặng cây gốc, rồi DFS xử lý từ dưới lên từng chuỗi nặng, giả sử mỗi cây con nhẹ đã được hợp nhất thành một cluster. Với mỗi đỉnh trên chuỗi nặng, lấy các cluster của những cây con nhẹ của cha đỉnh này, dùng chia để trị để *rake* chúng lại; cluster tổng nhận được lại được *rake* một lần nữa vào cạnh cha của đỉnh này. Để bảo đảm độ sâu không quá lớn, mỗi lần chia đôi, *rake* xong hai nửa rồi mới *rake* thành một cluster. Cây do thao tác này tạo ra gọi là *rake tree*. Sau đó ta thu được các cluster xếp thành một chuỗi, rồi *compress* chúng thành một cluster; tương tự cần dùng chia để trị để không chế độ sâu. Cây nhận được gọi là *compress tree*.

Bổ đề 1.3. Nếu chọn điểm giữa có trọng số theo kích thước cluster để chia để trị, thì Top Tree xây ra theo cách này có độ sâu $\Theta(\log n)$.

Dù là rake tree hay compress tree, cứ nhảy lên trên một số tầng hằng số thì kích thước cluster ít nhất nhân đôi, nên độ sâu là $O(\log n)$.

Sau đây ta bàn việc hợp nhất Top Tree được xây bằng thuật toán trên.

Ví dụ 1.3. Định nghĩa một thao tác: chọn một đoạn $[l, r]$, rồi đổi mọi ký tự trong đoạn từ ngoặc vuông sang ngoặc tròn và từ ngoặc tròn sang ngoặc vuông.

Định nghĩa trọng số $\text{val}(S)$ của một xâu ngoặc: nếu xâu ngoặc này có thể biến thành hợp lệ bằng các thao tác trên, trọng số là số thao tác ít nhất; nếu không thì là 0.

Cho q lần sửa đổi và truy vấn, có hai loại:

1. sửa đổi: cho đoạn $[l, r]$, đổi mọi ký tự trong đoạn từ ngoặc vuông sang ngoặc tròn và ngược lại;
2. truy vấn: cho đoạn $[l, r]$, tính

$$\sum_{[l', r'] \subseteq [l, r]} \text{val}(s[l', r']).$$

Bài này cần quan sát và biến đổi sơ bộ trọng số trong đề; phần ấy không liên quan nhiều tới bài viết nên được lược qua. Cấu trúc dữ liệu cần hỗ trợ: với một chuỗi đỉnh dọc trong cây, từ nông tới sâu là $a_1, a_2, \dots, a_k$, tính thông tin nửa nhóm của các cây con trong những con của a_i có chỉ số lớn hơn a_{i+1} , đồng thời hỗ trợ sửa đổi chuỗi trong cây.

Vì các thao tác không có tính chất thuận lợi, ta xét xây static Top Tree đầy đủ. Do thứ tự giữa các con là quan trọng trong bài này, mỗi nút compress cần nối hai cây con rake, lần lượt biểu thị thông tin các con nhẹ bên trái và bên phải của con nặng.

Khi định vị trên rake tree hoặc compress tree, cần chú ý các trường hợp sau. Giả sử cần định vị đoạn giữa u và v :

1. một trong u, v là tổ tiên của đỉnh kia;
2. không đỉnh nào trong u, v là tổ tiên của đỉnh kia;
3. u, v lần lượt nằm trong hai rake tree trái phải của con nặng;
4. một trong u, v là con nặng, tức không nằm trong rake tree;
5. u, v nằm trong cùng một rake tree.

Năm trường hợp này mô tả các tình huống gặp khi định vị trong static Top Tree; cách tính đóng góp cho từng loại không khó, nên bài viết không triển khai chi tiết. Thời gian là $O(n \log n)$, bộ nhớ $O(n)$.

Tương tự cây đoạn mở nút động, ta có thể có static decomposition tree mở nút động. Mỗi lần sửa đổi, ta định vị tới $O(\log n)$ nút bị ảnh hưởng, rồi xây các nút ấy và toàn bộ con của chúng. Vì mỗi nút hoặc là lá, hoặc có đủ mọi con, nên miễn là không đẩy tag xuống lá, số nút của cây sẽ không đổi.

Quá trình định vị khác với tìm kiếm từ gốc xuống của cây đoạn: chỉ cần ghi lại điểm biên tương ứng của cluster hiện tại. Ta không thể chịu chi phí mỗi lần tìm điểm biên mới, vì điều đó cần một phép nhị phân $O(\log n)$. Do đó ở đây thường xây trước một static Top Tree đầy đủ; khi định vị, duy trì đồng thời vị trí trên cây mở nút động và trên cây đầy đủ, rồi nhờ cây đầy đủ biết nút cần định vị nằm ở đâu. Cụ thể, trước hết định vị tới hai đầu của hai chuỗi sửa đổi, tức các đầu mút của phép sửa chuỗi. Sau đó liên tục nhảy lên cha trong static Top Tree đầy đủ và ghi lại thuộc về con nào. Chỉ cần một mảng phụ $O(\log n)$ để ghi đường đi. Rồi từ gốc đi xuống theo hai đường đi, vừa đi vừa mở nút động. Như vậy, với chi phí bộ nhớ phụ $O(\log n)$, ta giải quyết vấn đề định vị với hằng số nhỏ.

Sau khi định vị xong, tùy yêu cầu ta đánh tag chuỗi, tag cây con hoặc thực hiện truy vấn.

Bây giờ đã có static Top Tree mở nút động, xét cách hợp nhất hai static Top Tree mở nút động. Ta vẫn hợp nhất T_1, T_2 từ trên xuống.

Nếu không có lazy tag, có thể dùng đệ quy thô bạo rất đơn giản trên các nút ở vị trí tương ứng, và trả về tại nút rỗng. Ở đây thông tin khi hợp nhất cũng có vấn đề giống cây đoạn: cần chọn một trong hai cách duy trì thông tin. Giả sử có thể hợp nhất thông tin trong $O(1)$, ta nhận được thuật toán $O(n \log n)$.

Nếu có lazy tag, ta phải đối mặt với việc hợp nhất một nút lá và một cây con. Khi ấy trên nút lá có hai loại tag: tag trên đường giữa các điểm biên của cluster và tag trên toàn cluster trừ đường đó. Trường hợp hợp thường gặp là có thể đánh thẳng hai loại tag này lên nút tương ứng của static Top Tree bên kia. Nếu không được, cần thiết kế một thể năng cho hai bộ thông tin: thông tin trên đường giữa các điểm biên của cluster, và thông tin của toàn cluster ngoài phần đường đó. Vì hai bộ thông tin có liên hệ với nhau, cụ thể thông tin đường giữa các điểm không biên duy trì trên compress tree là tổng của hai phần trên rake tree, nên khi dùng thể năng và đệ quy thô bạo, cần chuyển đổi giữa hai loại thông tin.

Ví dụ 1.4. Cho một cây có gốc 1, gồm n đỉnh. Mỗi cạnh có một ký tự $c \in \{0, 1\}$. S_u là xâu tạo bởi việc viết lần lượt các ký tự trên đường từ gốc tới u . Bảo đảm các cạnh từ cùng một đỉnh tới các con có ký tự đôi một khác nhau.

Với mỗi đỉnh u , có một giá trị val_u và một giới hạn a_u . Với mỗi đỉnh u , cần tính

$$ans_u = \max_{\substack{0 \leq i \leq |S_u| \\ i \geq a_u, S_u = S_v[|S_v| - |S_u| + 1, |S_v|] \\ S_u[1, i] = S_v[1, i]}} val_v.$$

Xây automaton AC trên cây trie đầu vào. Bài toán chuyển thành: với mỗi nút u trên automaton AC, duy trì một cây trie có trọng số đỉnh. Cần hỗ trợ truy vấn tổng trên chuỗi và lấy max trên chuỗi. Khi hợp nhất hai cây trie, lấy max trọng số ở các nút tương ứng.

Vì đề bảo đảm cây là nhị phân và không có sửa/truy vấn cây con, ta có thể lược bỏ rake tree và việc duy trì thông tin đường giữa các điểm không biên.

Xét tới việc lấy max trên đoạn và hỏi tổng đoạn, đây là ứng dụng kinh điển của segment tree beats. Vì vậy ở mỗi nút duy trì giá trị nhỏ nhất, số lượng giá trị nhỏ nhất, giá trị nhỏ thứ hai và tổng đoạn; đồng thời duy trì tag lười biểu thị toàn bộ giá trị nhỏ nhất đã được cộng thêm bao nhiêu. Khi làm sửa chuỗi, trước hết định vị tới nút, rồi mô phỏng beats: nếu giá trị nhỏ thứ hai không nhỏ hơn val, thì xử lý bằng cắt tĩa; nếu không thì đệ quy theo thông tin sau đó. Để chứng minh phần này có thời gian $O(n \log n)$.

Với trường hợp hợp nhất cây, vẫn dùng khung hợp nhất static Top Tree tổng quát ở trên. Điểm duy nhất cần xử lý là khi hợp nhất một nút lá với một cây con static Top Tree: khi đó tương đương với sửa các nút trên compress tree bằng phép lấy max với val, trong đó val là tag cộng trên lá. Phép sửa lấy max này có thể tiếp tục dùng phương pháp beats. Vì thế bài toán được giải trong $O(n \log n)$ thời gian và $O(n)$ bộ nhớ.

1.5. Tách cây đoạn

Tách cây đoạn hỗ trợ tách một cây đoạn thành hai phần có miền giá trị không giao nhau. Chú ý rằng tách cây đoạn và hợp nhất cây đoạn không phải là hai phép ngược nhau.

Cụ thể, nếu cần chia dãy a_i thành

$$b_i = \begin{cases} a_i, & i \leq p, \\ 0, & i > p, \end{cases} \quad c_i = \begin{cases} 0, & i \leq p, \\ a_i, & i > p, \end{cases}$$

đồng thời duy trì thông tin đoạn của b_i, c_i , ta có thể thực hiện bằng cách tách cây đoạn đang duy trì dãy a_i .

Nếu một nút của cây đoạn biểu diễn đoạn $[l, r]$ thỏa $r \leq p$, thông tin của đoạn này có thể được sao chép nguyên vẹn sang nút tương ứng trong cây đoạn của dãy b . Tương tự, nếu $l > p$, thông tin đoạn có thể sao chép sang cây đoạn của dãy c . Các nút không thỏa cả hai điều kiện hiển nhiên chỉ có $O(\log n)$ nút; ta sao chép chúng thành hai bản và đặt vào vị trí tương ứng.

Cách làm này chỉ tăng thêm $O(\log n)$ nút, nên phân tích thể năng ban đầu vẫn đúng.

Ví dụ 1.5. Duy trì một dãy $a_1, a_2, \dots, a_n$ gồm các số nguyên không âm, hỗ trợ ba loại thao tác:

1. cho đoạn $[l, r]$, xor mọi số trong đoạn với x ;
2. cho đoạn $[l, r]$, sắp xếp các số trong đoạn theo thứ tự tăng dần;
3. cho đoạn $[l, r]$, hỏi xor của các số trong đoạn.

Vì thao tác sắp xếp đoạn khó duy trì trực tiếp, ta dùng phương pháp amortized theo các đoạn màu để né nó. Tại mọi thời điểm, viết a thành dạng: với mỗi tập S_i , sắp xếp các phần tử trong S_i , lần lượt xor với x_i , rồi nối các đoạn lại.

Dùng một 01-trie, tức cây đoạn trên miền giá trị, để duy trì tập S_i ; dùng một cây đoạn để duy trì toàn cục các x_i và tổng xor của mỗi tập sau khi xor với x_i . Mỗi khi gặp một đoạn thao tác $[l, r]$, tách các tập cắt qua $l-1, l$ và qua $r, r+1$. Ở đây cần hỗ trợ tách trie.

Thao tác xor đoạn chỉ cần sửa đoạn trên các x_i . Thao tác sắp xếp đoạn cần đánh tag lên các trie này rồi hợp nhất chúng. Truy vấn đoạn được thực hiện trên cây đoạn hỏi tổng xor đoạn. Do đó trong 01-trie chỉ cần duy trì kích thước cây con và xor của cây con.

Vì cần tách, không thể xây sẵn cây thao tác. Nhưng mọi sửa đổi trong bài này đều là sửa điểm, nên có thể dùng cây đoạn mở nút động dạng nén để đưa bộ nhớ về $O(n)$. Thời gian là

$$O(n(\log n + \log v)).$$

1.6. Tách thêm các cấu trúc dữ liệu tĩnh

Sau khi tách cấu trúc dữ liệu, việc lấy thông tin từ con hợp nhất lên là đơn giản. Nhưng nếu muốn tách trực tiếp từ thông tin cũ, thông tin được duy trì bắt buộc phải hỗ trợ tách. Vì vậy việc tách cây phân trị đỉnh/cạnh yêu cầu cao đối với thông tin và phạm vi ứng dụng hẹp, nên không phải trọng tâm của bài viết.

Xét tách một cây con và phân bù cây con trên static Top Tree.

Giả sử cần tách cây con của u . Ta có thể định vị phần cây con của u tương ứng trên static Top Tree: đó là các nút trong compress tree nơi u nằm có độ sâu phía sau u , cùng với các cây con rake tree của chúng. Phần này có thể được tách nguyên vẹn. Với các nút tổ tiên của u trên static Top Tree, do một phần của chúng nằm trong cây con của u trên cây gốc, phần còn lại không nằm trong đó, nên cần sao chép sang hai cây. Mỗi lần chỉ tạo mới $O(\log n)$ nút.

Ví dụ 1.6. Cho một cây T . Duy trì nhiều đa tập S_i , các phần tử thuộc $\{1, \dots, n\}$. Hỗ trợ:

1. tách mọi phần tử trong S_p có chỉ số nằm trong cây con của u sang S_q ;
2. với mọi đỉnh x trên đường từ u tới v , chèn x vào S_p đúng k lần;
3. hợp nhất hai tập;
4. hỏi tổng số lần xuất hiện trong S_p của mọi đỉnh x trên đường từ u tới v .

Dùng static Top Tree duy trì số lần xuất hiện của mỗi số. Sửa đổi tương đương cộng trên chuỗi, truy vấn tương đương hỏi tổng trên chuỗi, đồng thời cần hỗ trợ hợp nhất và tách.

Hợp nhất chỉ cần cộng theo vị trí, nên hợp nhất lá và cây con rất đơn giản: đánh lazy tag trực tiếp. Khi tách, trước hết định vị tới compress tree chứa u , rồi theo vị trí của u tách nút hiện tại thành hai phần và chọn một phía để đệ quy.

Thiết kế thế năng

$$\Phi = \sum_i |T_i|,$$

tức tổng số nút của các static Top Tree. Có thể phân tích được thời gian là $O(n \log n)$.

Sau khi xử lý tách cây con, một ý tưởng khác là xử lý tách đường. Nghĩa là sao chép một phần compress tree trong static Top Tree sang vị trí tương ứng của T' , nhưng các rake tree nối với những compress tree này không nên được sao chép sang vị trí tương ứng của T' ; chúng phải được giữ ở vị trí cũ. Do đó các nút T' cần sao chép không có dạng một số cây con; chi phí thô bạo bóc các rake tree nối với compress tree là không chấp nhận được.

Tuy vậy, gặp bài toán tách đường không có nghĩa là hết cách. Xét ví dụ sau.

Ví dụ 1.7. Cho một cây T . Duy trì nhiều đa tập S_i , các phần tử thuộc $\{1, \dots, n\}$. Hỗ trợ:

1. tách mọi phần tử trong S_p có chỉ số nằm trên đường từ u tới v sang S_q ;
2. với mọi đỉnh x trên đường từ u tới v , chèn x vào S_p đúng k lần;
3. hợp nhất hai tập;
4. hỏi tổng số lần xuất hiện trong S_p của mọi đỉnh x trên đường từ u tới v .

Vì chính cấu trúc của static Top Tree gây ra vấn đề trên, ta cần mở rộng thành một cấu trúc mới. Bài này không cần duy trì thông tin cây con, nên thông tin trong rake tree con của một compress node không quan trọng. Nói cách khác, không nhất thiết để rake tree làm con của compress node hiện tại.

Xét duy trì static Top Tree hai tầng: tách mỗi nút ban đầu thành u và u' , lần lượt biểu thị tầng thứ nhất và tầng thứ hai. Cấu trúc bên trong tầng thứ nhất giống static Top Tree thường. Bên trong tầng thứ hai chỉ xây compress tree. Với mỗi gốc rt của một compress tree, có thêm một cạnh trở tới rt' .

Chỉ tầng thứ hai duy trì thông tin chuỗi; tầng thứ nhất chỉ dùng để định vị và cân bằng trọng số. Rõ ràng cấu trúc trên có số con của mỗi nút là $O(1)$ và chiều cao $O(\log n)$.

Với static Top Tree hai tầng mở nút động, định vị như sau. Trước hết xây một static Top Tree đầy đủ, rồi định vị các nút tương ứng với đường u tới v trên đó. Sau đó tìm các nút tương ứng của những vị trí ấy trong static Top Tree tầng thứ hai, rồi mở nút động tới các nút được định vị và tổ tiên của chúng, bao gồm cả các nút ở tầng thứ nhất.

Khi đã có static Top Tree hai tầng mở nút động, ta có thể hợp nhất và hỗ trợ tách ra một đường. Thao tác hợp nhất tương tự static Top Tree thường. Với thao tác tách, đường cần tách nằm trên một số đoạn của các compress tree ở tầng thứ hai; vì compress tree tầng thứ hai không chứa rake tree dư thừa, các cây con này có thể được sao chép trực tiếp thành các nút tương ứng của T' . Tổng số nút dùng chung trên những compress tree này, kể cả các nút dùng chung trên static Top Tree tầng thứ nhất, là $O(\log n)$, nên có thể tạo mới chúng. Các nút khác vẫn giữ thông tin ngoài đường.

Thiết kế thế năng

$$\Phi = \sum_i |T_i|,$$

tức tổng số nút của các static Top Tree. Có thể phân tích được thời gian là $O(n \log n)$.

2. Hợp nhất và tách cấu trúc dữ liệu động

2.1. Dẫn nhập

Cấu trúc dữ liệu động có phạm vi ứng dụng rất rộng; do đặc điểm động, chúng có thể ứng phó với nhiều loại thao tác. Nhưng chính sự bất định của cấu trúc cũng làm việc hợp nhất trở nên khó khăn. Hợp nhất heuristic là cách ứng phó phổ biến nhất, nhưng vì có thêm một thừa số log, lại không hỗ trợ tách, nên chưa thỏa đáng. Phần sau giới thiệu một lớp bài toán trên cây cân bằng.

2.2. Hợp nhất cây cân bằng

Ví dụ 2.1. Cần duy trì nhiều dãy số nguyên tăng dần, hỗ trợ:

1. cộng mọi phần tử trong một dãy với cùng một hằng số;
2. đổi dấu mọi phần tử trong một dãy và đảo ngược cả dãy;
3. trộn hai dãy thành một dãy;
4. hỏi tổng một đoạn của một dãy.

Có nhiều loại cây cân bằng, và cách hợp nhất cũng khác nhau. Bài viết chỉ giới thiệu hợp nhất treap. Trước hết chứng minh một số bổ đề về treap.

Bổ đề 2.1. Treap có n nút có độ sâu kỳ vọng $O(\log n)$.

Giả sử sau khi sắp xếp mọi phần tử theo key, các giá trị ngẫu nhiên lần lượt là $b_1, b_2, \dots, b_n$ (không mất tổng quát, giả sử mọi phần tử đôi một khác nhau). Với x và y , xét hai trường hợp:

1. nếu $x \leq y$, thì x là tổ tiên của y khi và chỉ khi

$$b_x < \min_{x < j \leq y} b_j;$$

2. nếu $x > y$, thì x là tổ tiên của y khi và chỉ khi

$$b_x < \min_{y \leq j < x} b_j.$$

Xác suất của sự kiện này chính là xác suất x có giá trị ưu tiên nhỏ nhất, tức

$$\frac{1}{|x - y| + 1}.$$

Độ sâu của x có thể định nghĩa là số tổ tiên của x , nên kỳ vọng số tổ tiên là

$$\sum_{i=1}^n \frac{1}{|x - i| + 1} \sim O(\log n).$$

Định nghĩa 2.1. *Finger search* là thao tác trong một cấu trúc dữ liệu mà nếu gọi $d(x, y)$ là số phần tử giữa x và y (bao gồm x, y), thì sau khi đã xác định vị trí của x , có thể truy cập y trong thời gian

$$O(f(d(x, y))).$$

Hiển nhiên nếu $f(x) > \log x$ thì không có ý nghĩa, vì tìm kiếm lại từ đầu còn tốt hơn. Vì vậy phần sau mặc định $f(x) = \log x$.

Bổ đề 2.2. Trên treap, độ dài đường đi từ x tới y có kỳ vọng

$$O(\log d(x, y)).$$

Đường đi từ x tới y có độ dài bằng: số nút là tổ tiên của x nhưng không là tổ tiên của y , cộng với số nút là tổ tiên của y nhưng không là tổ tiên của x . Theo đối xứng, chỉ cần xét về đầu.

Giả sử mọi phần tử của treap theo thứ tự tăng dần là

$$a_1, a_2, \dots, a_n,$$

trong đó $a_i = x, a_j = y$. Rõ ràng $d(x, y) = j - i + 1$. Xét một nút a_k và tính xác suất nó thỏa “là tổ tiên của x nhưng không là tổ tiên của y ”, rồi cộng lại. Ta phân loại theo k :

1. $1 \leq k \leq i$. Điều này tương đương k là giá trị nhỏ nhất trên $[k, i]$, và giá trị nhỏ nhất trên $[k, i]$ lớn hơn giá trị nhỏ nhất trên $(i, j]$. Xác suất phần trước là $1/(i - k + 1)$, phần sau là $(j - i)/(j - k + 1)$. Tích lại được

$$\frac{j - i}{(i - k + 1)(j - k + 1)} = \frac{1}{i - k + 1} - \frac{1}{j - k + 1}.$$

Cộng lại cho k cho ra $O(\log d(i, j))$.

2. $i < k < j$. Tương tự, xác suất là

$$\frac{j - k}{(k - i + 1)(j - i + 1)} = \frac{1}{k - i + 1} - \frac{1}{j - i + 1}.$$

Cộng lại cũng được $O(\log d(i, j))$.

Từ đó thấy phương pháp finger search trên treap là: từ x liên tục nhảy lên cho tới $\text{lca}(x, y)$, rồi đi xuống định vị tới y .

Xét hợp nhất hai treap T_1, T_2 , kích thước lần lượt là n, m . Không mất tổng quát giả sử $m \leq n$. Gọi

$$p_1 < p_2 < \dots < p_m$$

là thứ hạng của các phần tử trong T_2 giữa $n + m$ phần tử. Nếu chèn lần lượt theo thứ tự, thời gian là

$$O\left(\sum_{i=1}^m \log(p_i - p_{i-1})\right), \quad p_0 = 0.$$

Do tính lồi của $\log$, giá trị lớn nhất của biểu thức trên là

$$O\left(m \log \frac{n}{m}\right).$$

Thiết kế thế năng

$$\Phi = \sum_i |T_i| \log |T_i|.$$

Mỗi lần hợp nhất làm thế năng thay đổi

$$\begin{aligned} & (n + m) \log(n + m) - n \log n - m \log m \\ & \geq m \log(n + m) - m \log m \geq m \log \frac{n}{m}. \end{aligned}$$

Tức ta dùng thời gian $O(m \log(n/m))$ để làm thế năng tăng ít nhất $m \log(n/m)$. Vì thế năng có chặn trên $n \log n$, tổng thời gian là $O(n \log n)$.

2.3. Tách cây cân bằng

Ví dụ 2.2. Cần duy trì nhiều dãy số nguyên tăng dần, hỗ trợ:

1. với một dãy a , cho trọng số k , tách a thành hai dãy gồm các phần tử nhỏ hơn k và các phần tử không nhỏ hơn k ;
2. cộng mọi phần tử trong một dãy với cùng một hằng số;
3. đổi dấu mọi phần tử trong một dãy và đảo ngược cả dãy;
4. trộn hai dãy thành một dãy;
5. hỏi tổng một đoạn của một dãy.

Tách cây cân bằng cũng nghĩa là chia theo miền giá trị thành hai phần không giao nhau. Với treap, việc này có thể xử lý trong $O(\log n)$.

Với thao tác hợp nhất, ta xét trộn thô bạo hai dãy. Mỗi lần lấy phần tử đầu dãy, tức giá trị nhỏ nhất; giả sử là $x \leq y$. Sau đó tách khỏi T_1 mọi phần tử có giá trị $\leq y$; lúc này nhất định có $x > y$, rồi tách khỏi T_2 mọi phần tử có giá trị $\leq x$. Lặp lại như vậy cho tới khi hoàn tất trộn. Tiếp theo chứng minh độ phức tạp của cách làm này.

Với một cây T , viết trọng số các nút theo thứ tự tăng dần là

$$a_1, \dots, a_n.$$

Xét thể năng

$$\Phi = \sum_T \sum_{i=1}^{n+1} \log(a_i - a_{i-1}),$$

trong đó $a_0 = 0$, $a_{n+1} = v$, lần lượt biểu thị cận dưới và cận trên của miền giá trị.

Ta vẫn dùng cách làm của phần trước. Đặt

$$k = \sum_i [c_i \neq c_{i+1}],$$

trong đó $c_i \in \{0, 1\}$ biểu thị a_i thuộc T_1 hay T_2 . Từ đó có thể chứng minh thời gian hợp nhất hai cây là $O(k \log n)$, còn thể năng giảm $O(k)$. Φ có chặn trên $O(n \log v)$, và các thao tác sửa đổi không ảnh hưởng tới thể năng, nên tổng thời gian là

$$O(n \log n \log v).$$

Tổng kết

Bài viết giới thiệu việc hợp nhất và tách cấu trúc dữ liệu, chia theo cấu trúc tĩnh và động. Phần tĩnh giới thiệu tư tưởng hợp nhất và tách một lớp cấu trúc dữ liệu tĩnh: xuất phát từ cây đoạn, rồi mở rộng sang cây phân trị đỉnh, cây phân trị cạnh và static Top Tree; đồng thời đưa ra thuật toán hợp nhất và tách trên static Top Tree. Phần động giới thiệu cách cây cân bằng xử lý nhược điểm do tính động gây ra, và từ góc nhìn finger search giải quyết nhiều bài toán hợp nhất cây cân bằng. Các kỹ thuật này có triển vọng ứng dụng rộng trong thi tin học.

Tuy nhiên, trong lý thuyết cấu trúc dữ liệu vẫn còn nhiều vấn đề bỏ ngỏ. Tác giả hy vọng bài viết có thể gợi mở để độc giả tiếp tục nghiên cứu sâu hơn các lý thuyết liên quan và làm chúng trưởng thành hơn.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Ye Jinyi, Liang Lei và Gu Duoyu của Trường Trung học trực thuộc Đại học Nhân dân Trung Quốc đã quan tâm và hướng dẫn.

Cảm ơn gia đình và bạn bè đã ủng hộ, động viên tác giả.

Cảm ơn các bạn Lou Yuzhan và Hu Zhaoyang đã thảo luận và đem lại gợi ý cho tác giả.

Cảm ơn các bạn Lou Yuzhan và Ye Liqi đã đọc duyệt bài viết.

Tài liệu tham khảo

1. Cheng Siyuan. *Bàn về ứng dụng của static Top Tree trên cây và đồ thị series-parallel tổng quát*. Tập luận văn đội tuyển quốc gia, 2023.
2. Zhao Yuyang. *Lý thuyết, cách dùng và cài đặt các nội dung liên quan tới Top tree*. <https://uoj.ac/blog/4912>.
3. Wikipedia. *Finger search*. https://en.wikipedia.org/wiki/Finger_search.
4. Hu Zhaoyang. *Cây cân bằng và finger search*. cnblogs.com/PYWBKTDA/p/14576937.
5. Zhou Xin. *Về một bài toán liên quan tới cây cân bằng*. <https://www.luogu.com.cn/blog/user19567/guan-yu-yi-ge-ping-heng-shu-xiang-guan-di-wen-ti>.

3 Chủ đề chính

Từ mục lục, tập năm 2024 bao phủ các nhóm chủ đề sau:

- hàm nhân tính, số học, liên phân số, thuật toán Euclid và q -analogue;
- matroid tuyến tính, matching có trọng số đỉnh và tô màu danh sách;
- cấu trúc dữ liệu, cập nhật-truy vấn đoạn và các bài toán trên cây;
- quy hoạch động, hợp nhất thông tin, bao hàm-loại trừ và nghịch đảo;
- hàm sinh, quá trình ngẫu nhiên, đếm đường đi trên lưới;
- tối ưu hằng số trên phần cứng hiện đại, lý thuyết tính toán và độ phức tạp;
- lattice, phân tích đa thức hệ số nguyên, chu trình tỉ lệ nhỏ nhất và các báo cáo ra đề.

4 Ghi chú về OCR

Phần mục lục của tập 2024 trích xuất được rõ ràng, nhưng lớp văn bản thân bài không ổn định: nhiều đoạn trở thành mojibake và công cụ PDF báo cảnh báo phông chữ. Bài đầu tiên ở trên được dịch từ OCR tại `/tmp/oipl-ocr/2024-p9-18.txt` và đối chiếu với ảnh render các trang PDF 9–18. Bài thứ hai được dịch từ lớp văn bản PDF và OCR tại `/tmp/oipl-2024-parity/`, đối chiếu với ảnh render PDF trang 19–31, tương ứng trang in 11–23. Một số công thức dài trong phần 4 được giữ ở mức có thể xác nhận từ ảnh trang; các biến phụ như $\omega(m)$ được hiểu theo ký hiệu chuẩn là số lượng thừa số nguyên tố phân biệt. Bài thứ ba được dịch từ OCR và lớp văn bản PDF tại `/tmp/oipl-2024-ds/`, đối chiếu với ảnh render PDF trang 32–44, tương ứng trang in 24–36; hình minh họa Top Tree trong nguồn được diễn giải bằng lời thay vì vẽ lại.